

MAMBA-NIKE

Specification for a Non-Interactive Key Exchange from Ring Learning with Quantization

Shanxiang Lyu¹ Hao Cheng¹ Tang Zhou¹ Ke Ma¹
Chengjing Tian¹ Junzuo Lai¹ Zheng Wang² Jinming Wen³
Tao Yang⁴ Zheng Qin⁵ Zhengchun Zhou⁶

¹ College of Cyber Security, Jinan University, Guangzhou, China

² Southeast University, Nanjing, China

³ Jilin University, Changchun, China

⁴ Beihang University, Beijing, China

⁵ Shenzhen Institute of Beihang University, Shenzhen, China

⁶ Southwest Jiaotong University, Chengdu, China

June 29, 2026

Abstract

MAMBA-NIKE is a family of ring-based non-interactive key-exchange mechanisms built from Ring Learning with Quantization over $\mathbb{Z}$ (RLWQ-Z) and NewHope-style D_4 reconciliation. After public-key distribution, the protocol needs only one online message: the initiator sends $M_1 = (\mu, u, h)$, and both parties derive a shared secret from two close ring products. The scheme is submitted as an NGCC KEX primitive and targets passive real-or-random key-exchange security.

MAMBA-NIKE uses public dithered coefficient-wise quantization as a native transport operation, rather than as a separate compression step. Each transmitted split label has a public compensated normal form, and the resulting quantization error is bounded and independent of the hidden input value. This allows the public key and the one-pass response to be analyzed through an RLWE-type normal form, while keeping the transmitted objects compact. The D_4 reconciliation layer then converts the two close agreement values into a common raw reconciliation string, which is bound to the transcript by the submitted SHAKE256 key-derivation function.

All submitted profiles use $q = 2^{13}$, $n \in \{1024, 2048\}$, power-of-two compression moduli, and p-packed external encodings. Coefficient reduction is implemented by masking. The Reference implementation uses coefficient-domain negacyclic Toom–Cook-4 convolution, while the x86-64 Optimized implementation uses a fixed-iteration AVX2 multiplier for products with one small CBD operand and retains Toom–Cook-4 as its portable fallback; neither track uses a prime-field NTT. This document specifies five profiles for 128, 192, 256, 384, and 512-bit classical security targets, together with parameter choices, key-acceptance analysis, security estimates, performance records, known-answer tests, and implementation requirements.

Contents

1	Introduction	4
1.1	Design Rationale	5
2	General Algorithm Specification	7
2.1	Notation	7
2.2	The MAMBA-NIKE.KE Key Exchange	7
3	Parameter Settings	9
3.1	Selected Parameter Sets	9
3.2	Space Overheads and External Byte Lengths	10
3.3	Correctness and Security Summary	10
4	Performance Evaluation	11
4.1	Experimental Setup and Platforms	11
4.2	Implementation Variants	11
4.3	x86 Implementation Results	12
4.4	Comparison to NewHope KE	12
5	Known Answer Tests	14
6	Security Strength	14
6.1	Security Definitions and Claims	15
6.2	Theoretical Security	16
6.3	Classical and Quantum Security Levels	17
6.4	Side-Channel Security Considerations	17
7	Advantages and Limitations	18
7.1	Advantages	18
7.2	Limitations	19
8	Implementation Specification	20
8.1	External KEX API and Implementation Boundary	20
8.2	Integer Representation	21
8.3	Serialized Objects	21
8.4	Bit Packing and Parsing	22
8.5	Seed Expansion and Dither Generation	22
8.6	Secret Sampling	22
8.7	Dithered Quantization and Compensation	23
8.8	Polynomial Arithmetic	23
8.9	D_4 Reconciliation	24
8.10	Constant-Time Requirements	24
8.11	Validation Requirements	24
A	Mathematical Foundations	25
A.1	The RLWQ Problem	25
A.2	Centered Binomial Distributions	26
A.3	Arithmetic Layer	26

B	Security Proof Details	27
B.1	Split and Normal RLWQ- $\mathbb{Z}$ Samples	27
B.2	Transport Samples in the KE Transcript	28
B.3	Passive Real-or-Random Security of MAMBA-NIKE	28
B.4	Scope of Active and CCA-Style Claims	29
C	Key-Acceptance Error Analysis	29
C.1	Correctness and Key-Acceptance Analysis	29

1 Introduction

MAMBA-NIKE is a ring-based non-interactive key exchange (NIKE) mechanism designed for post-quantum cryptography standardization. The underlying scheme is built over the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ and uses Ring Learning with Quantization over $\mathbb{Z}$ (RLWQ – $\mathbb{Z}$) [2] as its transport layer, a hardness assumption that natively bridges the compact communication performance of rounding-based constructions with the rigorous, transparent security reductions of error-based lattice cryptography.

MAMBA-NIKE shares with NewHope [10] the same high-level structure: both are ring-based one-pass key-exchange protocols after public-key distribution that use D_4 reconciliation to convert approximately equal ring products into a common shared secret. However, MAMBA-NIKE differs from NewHope in several fundamental aspects that together define its design identity:

- **Modulus arithmetic.** NewHope uses a prime modulus ($q = 12289$) to enable a native NTT over R_q . MAMBA-NIKE uses the power-of-two modulus $q = 2^{13}$, which eliminates prime-field modular reduction routines (coefficient reduction becomes a branchless mask operation) but forecloses the native prime-field NTT path. Polynomial multiplication in MAMBA-NIKE is therefore implemented by coefficient-domain Toom–Cook-4 convolution rather than by an NTT.
- **Error source.** NewHope follows the RLWE paradigm and samples explicit centered-binomial error polynomials for both the public key and the response. MAMBA-NIKE replaces explicit error sampling with public dithered quantization [2]: a public dither is added before quantization and subtracted during reconstruction, yielding an input-independent quantization error with a fully specified bounded distribution.
- **Compression paradigm.** In NewHope, bandwidth compression is achieved by auxiliary coefficient rounding applied after the core RLWE operations. In MAMBA-NIKE, quantization is a first-class cryptographic primitive: the transmitted split labels are quantization outputs, and their normal-form reconstructions are the algebraic values that enter multiplication. Compression is therefore not an auxiliary post-processing step but an integral part of the transport layer.
- **Security-reduction surface.** Because NewHope transmits explicit RLWE samples, its security reduction is directly to the decisional RLWE problem. MAMBA-NIKE transmits split RLWQ – $\mathbb{Z}$ samples (quantization label plus public dither seed) and justifies security through the reconstructed normal-form RLWE proxy. This separation between the transmitted split form and the normal-form analysis model is a distinctive feature of the RLWQ – $\mathbb{Z}$ framework.

Table 1 summarizes these structural distinctions concisely.

Table 1: High-level architectural comparison of MAMBA-NIKE with NewHope-KE.

Feature	NewHope-KE	MAMBA-NIKE
Underlying Problem	RLWE	RLWQ
Modulus Arithmetic	Prime (12289)	Power-of-two (2^{13})
Error Source	Explicit Sampling	Public Dithered Quantization
Noise Independence	Yes	Yes
Bandwidth Compression	Auxiliary Rounding	Native Quantization
Security Reduction	Direct to RLWE	Split-RLWQ / RLWE normal form

Table 2: Executive summary of MAMBA-NIKE profiles: p-packed external bandwidth records and concrete security margins.

Scheme	$ \text{pk} $ (B)	$ M_1 $ (B)	$ \text{sk} _{\text{API}}$ (B)	MATZOV (C)	MATZOV (Q)
MAMBA-NIKE-128	1184	1568	3232	272.1	251.1
MAMBA-NIKE-192	1312	1568	3360	279.3	258.4
MAMBA-NIKE-256	1312	1568	3360	294.3	273.9
MAMBA-NIKE-384	2848	3360	6944	538.6	512.5
MAMBA-NIKE-512	2848	3360	6944	590.2	512.5

The hierarchical architecture of the MAMBA-NIKE key exchange consists of three decoupled structural layers:

1. **RLWQ – $\mathbb{Z}$ Transport Layer:** A responder public key and an initiator one-pass message are represented as split dithered-quantization samples. Their reconstructed normal forms are analyzed as RLWE-type samples with explicit quantization-error distributions, inheriting the compactness of quantization while supporting a transparent RLWE-type normal-form analysis.
2. **D_4 Reconciliation Layer:** A NewHope-style four-dimensional reconciliation mechanism converts two close ring products into a common raw reconciliation string. The helper vector is public, compact (one byte per recovered key bit), and fixed to the submitted KAT format.
3. **Key Derivation and API Layer:** A transcript-bound SHAKE256 KDF derives a profile-dependent shared secret of the declared API length, and the NGCC KEX interface exposes deterministic length queries, key generation, one-pass message generation, and shared-secret derivation.

The operational execution pipeline of the KE proceeds through three core phases:

- **Public-key generation:** The responder samples a short secret polynomial s , derives a public polynomial a from a seed, and publishes a quantized public component b together with the seed.
- **One-pass message generation:** The initiator uses the responder public key, samples an ephemeral secret r , computes the response polynomial u , computes the local agreement value, generates the D_4 helper vector, and derives the initiator shared secret.
- **Responder derivation:** The responder reconstructs the response value from u , multiplies by its secret key, applies the received helper vector, and derives the same shared secret.

1.1 Design Rationale

This section justifies the foundational algebraic design choices, performance-security trade-offs, and mathematical rationales driving the implementation of MAMBA-NIKE.

1.1.1 The RLWQ – $\mathbb{Z}$ Transport Layer

The post-quantum lattice key-exchange landscape is heavily influenced by the Ring Learning with Errors (RLWE) paradigm, exemplified by NewHope [10]. RLWE-based constructions rely on explicitly sampled additive noise to achieve provable security. The noise does not inherently contribute to bandwidth compression; instead, it leads to more severe side-channel attacks [24, 25]. Consequently, compression in these schemes is treated as an

auxiliary post-processing step—coefficient rounding or bit-dropping applied after the core algebraic operations—rather than as an integral part of the hardness assumption.

By deploying the RLWQ $-\mathbb{Z}$ formulation, MAMBA-NIKE elevates quantization from an auxiliary compression tool to a first-class cryptographic primitive. The central enabling mechanism is public dithered quantization [2, 29]: a pseudo-random, publicly accessible dither is added before quantization and subtracted during reconstruction. In the divisible power-of-two setting used here, the resulting quantization noise becomes statistically independent of the hidden input data and has a fully specified bounded distribution. This algebraic independence enables a transparent normal-form analysis through RLWE-type samples, while the transmitted split form retains the compactness of quantization-based compression.

1.1.2 Power-of-Two Modulus

MAMBA-NIKE adopts the power-of-two modulus $q = 2^{13}$ across all submitted profiles. This algebraic structure provides substantial implementation advantages for both compression operations and underlying integer arithmetic. Most notably, coefficient reduction modulo q becomes a branchless mask operation, and coefficient-wise quantization collapses to standard bit-shifts. Consequently, expanding compressed labels into their normal forms requires only basic integer shifts, additions, and subtractions. Because all coefficient operations map directly to standard fixed-width unsigned integer types, MAMBA-NIKE eliminates the need for explicit, variable-time modular reduction routines, minimizing execution latency and enforcing constant-time compliance by default.

The primary constraint of a power-of-two modulus is the absence of a native invertible Number Theoretic Transform (NTT) over the primary cyclotomic ring, as the polynomial degree is not coprime to q . To bypass this bottleneck without compromising throughput, MAMBA-NIKE uses coefficient-domain negacyclic convolution: the implementation computes the full integer product using Toom–Cook-4 multiplication and folds the result modulo $X^n + 1$ [12, 13]. This keeps the reference arithmetic layer transparent and auditable while leaving room for optimized vectorized multiplication backends such as AVX2.

1.1.3 The D_4 Reconciliation Layer

MAMBA-NIKE retains the NewHope-style D_4 reconciliation mechanism because it is compact, well studied, and naturally suited to converting approximate ring products into a common raw key string. For each raw key bit, four coefficients are grouped into a reconciliation block and four two-bit helper values are packed into one byte. The submitted implementation consumes $\kappa = n/4$ reconciliation blocks, producing a κ -bit raw reconciliation string and an $n/4$ -byte packed helper vector in the one-pass message. Thus the helper is 256 bytes for $n = 1024$ profiles and 512 bytes for $n = 2048$ profiles.

This design cleanly separates the transport-error source from the reconciliation mechanism. The transport errors arise from public dithered quantization, while the reconciliation layer only requires that the accumulated agreement error remain inside the D_4 decoding radius. The resulting correctness expression,

$$\Delta = e_{\text{pk}}r - e_us + e_v,$$

makes parameter selection explicit: p_{pk} and p_u control both bandwidth and exposed quantization noise, while p_v controls the local reconciliation perturbation. This separation allows the security analysis to treat the D_4 helper as a public auxiliary string whose leakage is bounded by standard reconciliation arguments.

2 General Algorithm Specification

2.1 Notation

Let $R = \mathbb{Z}[X]/(X^n + 1)$ and $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. For any positive modulus p , write $R_p = \mathbb{Z}_p[X]/(X^n + 1)$. In the submitted parameter sets, $n \in \{1024, 2048\}$ and q is a power of two. Elements of R_q are represented by coefficient vectors in $\mathbb{Z}_q^n$; the coefficient embedding is

$$\tau(a_0 + a_1X + \cdots + a_{n-1}X^{n-1}) = (a_0, \dots, a_{n-1}).$$

Lowercase letters such as a, b, u, v, s, r denote ring elements unless explicitly stated otherwise. Bold lowercase letters such as $\mathbf{y}, \mathbf{d}, \mathbf{e}$ denote coefficient or lattice vectors, and bold uppercase letters such as $\mathbf{B}$ denote matrices or lattice bases. Byte-string and protocol objects use roman notation, for example $\mathbf{pk}, \mathbf{sk}, M_1, K_A$, and K_B .

For $x \in \mathbb{R}$, $\lfloor x \rfloor$ denotes nearest-integer rounding, $\lfloor x \rfloor_q$ denotes the canonical representative in $[0, q)$, and $\lfloor x \rfloor_q^\pm$ denotes the centered representative in $[-\lfloor q/2 \rfloor, \lfloor q/2 \rfloor]$. If S is a finite set, $x \leftarrow \mathcal{U}(S)$ denotes uniform sampling from S . If χ is a distribution, $x \leftarrow \chi$ denotes sampling according to χ . For a positive integer η , $\mathcal{B}_\eta$ denotes the centered binomial distribution obtained as $\sum_{i=1}^\eta (a_i - b_i)$ for independent uniform bits $a_i, b_i \leftarrow \{0, 1\}$. For ring samples, $\mathcal{B}_\eta$ is applied coefficient-wise.

For each quantized component $x \in \{\mathbf{pk}, u, v\}$, define

$$p_x = 2^{t_x}, \quad \Delta_x = q/p_x, \quad \Lambda_x = \Delta_x \mathbb{Z}^n.$$

The public-key component uses $p_{\mathbf{pk}}$ and $\Delta_{\mathbf{pk}}$; the online response component uses p_u and Δ_u ; the local reconciliation component uses p_v and Δ_v . Let $\mathcal{D}_x \subset R$ be the set of dither polynomials whose coefficients lie in $\{0, \dots, \Delta_x - 1\}$. The induced scalar quantization-error distribution for component x is denoted by $\chi_x = \chi_{q, p_x}$. For $a \in R_q$ and $d \in \mathcal{D}_x$, the split quantization label is written

$$\mathbf{Q}_{\Lambda_x, d}^\diamond(a) \in R_{p_x}.$$

If $c = \mathbf{Q}_{\Lambda_x, d}^\diamond(a)$ is such a label, its normal-form expansion is

$$\hat{c} = \Delta_x c - d \pmod{q}.$$

Equivalently, coefficient-wise nearest rounding can be implemented as

$$\mathbf{Q}_{q, p_x, d}(y) = ((y + d + 2^{h_x-1}) \gg h_x) \& (2^{t_x} - 1), \quad h_x = \log_2 q - t_x.$$

The notation $\hat{b}$, $\hat{u}$, and $\hat{v}$ always denotes the normal-form expansion of the corresponding split label.

2.2 The MAMBA-NIKE.KE Key Exchange

MAMBA-NIKE is submitted as a KE primitive. After public-key distribution, one online pass is sufficient: the initiator uses the responder's public key to produce M_1 and its shared secret, and the responder derives the same shared secret from M_1 and its secret key. This is the flow implemented by the NGCC KEX API, for which `kex_get_passes_num()` returns 1.

The normative flow is given by Algorithms 1, 2, and 3. The algorithms are written in split-label form: transmitted ring elements are quantization labels, while values marked with a hat are their normal-form reconstructions in R_q . The dither seed ρ belongs to the responder public key, and the online seed μ belongs to the one-pass message.

2.2.1 KE Key Generation

Algorithm 1: MAMBA-NIKE.KeyGen()

Output: Public key $\text{pk} = (\rho, b)$ and secret key $\text{sk} = s$

- 1 $\rho \leftarrow \mathcal{U}(\{0, 1\}^{256})$;
- 2 $(a, d_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$;
- 3 $s \leftarrow \mathcal{B}_{\eta_s}^n$;
- 4 $b \leftarrow Q_{\Lambda_{\text{pk}}, d_{\text{pk}}}^\diamond(as)$;
- 5 **return** $((\rho, b), s)$;

The seed ρ determines both the public polynomial a and the public-key dither d_{pk} through GenPublic . The secret key is the short polynomial s sampled from $\mathcal{B}_{\eta_s}$. The public component b is stored as a split quantization label in R_{pk} ; its associated normal form is reconstructed only when needed as $\hat{b} = \Delta_{\text{pk}}b - d_{\text{pk}} \pmod{q}$. This keeps the public key compact while retaining the additive normal-form view used in the security analysis.

2.2.2 KE Pass Generation

Algorithm 2: MAMBA-NIKE.Pass1(pk_B)

Input: Responder public key $\text{pk}_B = (\rho, b)$
Output: Pass message M_1 and initiator shared key K_A

- 1 $(a, d_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$;
- 2 $\hat{b} \leftarrow \Delta_{\text{pk}}b - d_{\text{pk}} \pmod{q}$;
- 3 $\mu \leftarrow \mathcal{U}(\{0, 1\}^{256})$;
- 4 $r \leftarrow \mathcal{B}_{\eta_r}^n$;
- 5 $(d_u, d_v) \leftarrow \text{GenDither}(\mu)$;
- 6 $u \leftarrow Q_{\Lambda_u, d_u}^\diamond(ar)$;
- 7 $v \leftarrow Q_{\Lambda_v, d_v}^\diamond(\hat{b}r)$;
- 8 $\hat{v} \leftarrow \Delta_v v - d_v \pmod{q}$;
- 9 $h \leftarrow \text{HelpRec}(\hat{v})$;
- 10 $\nu_A \leftarrow \text{Rec}(\hat{v}, h)$;
- 11 $M_1 \leftarrow (\mu, u, h)$;
- 12 $K_A \leftarrow \text{KDF}(\nu_A, \text{pk}_B, M_1)$;
- 13 **return** (M_1, K_A) ;

The initiator first reconstructs the responder public-key normal form $\hat{b}$ and then samples an ephemeral secret r . The online seed μ determines the response-side dithers d_u and d_v . The response label u is transmitted, while v is an initiator-local quantized value used only to compute $\hat{v}$, the reconciliation helper h , and the initiator raw reconciliation string ν_A . The pass message is $M_1 = (\mu, u, h)$: μ lets the responder regenerate the same response-side dithers, u is the split response label, and h is the public D_4 reconciliation helper. The final initiator key binds the raw reconciliation string to the responder public key and the full pass transcript through KDF.

Table 3: Submitted parameter profiles for MAMBA-NIKE. Here $p_{\text{pk}} = 2^{t_{\text{pk}}}$, $p_u = 2^{t_u}$, and $p_v = 2^{t_v}$. ρ_{key} reports the current numerical status of rejection probability.

Instance	Target	MATZOV-C	MATZOV-Q	n	q	t_{pk}	t_u	t_v	η_s, η_r	ρ_{key}
MAMBA-NIKE-128	128	272.1	251.1	1024	2^{13}	9	10	6	2,2	$2^{-42.44}$
MAMBA-NIKE-192	192	279.3	258.4	1024	2^{13}	10	10	6	3,3	$2^{-40.65}$
MAMBA-NIKE-256	256	294.3	273.9	1024	2^{13}	10	10	6	7,7	$2^{-39.97}$
MAMBA-NIKE-384	384	538.6	512.5	2048	2^{13}	11	11	6	2,2	$2^{-39.69}$
MAMBA-NIKE-512	512	590.2	512.5	2048	2^{13}	11	11	6	5,5	$2^{-37.81}$

2.2.3 KE Key Derivation

Algorithm 3: MAMBA-NIKE.Derive($\text{sk}_B, \text{pk}_B, M_1$)

Input: Responder secret key $\text{sk}_B = s_B$, public key pk_B , pass message

$M_1 = (\mu, u, h)$

Output: Responder shared key K_B

- 1 $(d_u, d_v) \leftarrow \text{GenDither}(\mu)$;
 - 2 $\hat{u} \leftarrow \Delta_u u - d_u \pmod{q}$;
 - 3 $w \leftarrow \hat{u} s_B$;
 - 4 $\nu_B \leftarrow \text{Rec}(w, h)$;
 - 5 $K_B \leftarrow \text{KDF}(\nu_B, \text{pk}_B, M_1)$;
 - 6 **return** K_B ;
-

The responder parses $M_1 = (\mu, u, h)$, regenerates d_u, d_v , and reconstructs the response normal form $\hat{u}$. It then computes $w = \hat{u} s_B$, which is close to the initiator's $\hat{v}$ whenever the total agreement error remains within the D_4 decoding radius. Applying the received helper h recovers the responder raw reconciliation string ν_B . The responder derives K_B using the same transcript binding as the initiator. Correctness is therefore reduced to showing that $\text{Rec}(\hat{v}, h) = \text{Rec}(w, h)$ except with the stated key-rejection probability.

3 Parameter Settings

This section outlines the definitive parameter configurations, structural byte sizes, and security/correctness summary for the MAMBA-NIKE. All submitted profiles use power-of-two arithmetic moduli and power-of-two quantization moduli. For each quantized component $x \in \{\text{pk}, u, v\}$, the compression modulus is $p_x = 2^{t_x}$ and the quantization step is $\Delta_x = q/p_x$.

3.1 Selected Parameter Sets

Let λ denote the claimed classical security target of a profile. All submitted profiles use $q = 2^\ell$ and power-of-two compression moduli $p_{\text{pk}} = 2^{t_{\text{pk}}}$, $p_u = 2^{t_u}$, and $p_v = 2^{t_v}$. The centered binomial parameters are profile-specific: $(\eta_s, \eta_r) = (2, 2), (3, 3), (7, 7), (2, 2), (5, 5)$ for the 128-, 192-, 256-, 384-, and 512-bit profiles, respectively. The reference KEX implementation outputs the profile-dependent shared-secret length listed in Table 4 by applying transcript-bound SHAKE256 to the raw reconciliation string, responder public key, and full one-pass message [11].

The profiles are selected by balancing three quantities: concrete attack cost, key-acceptance probability, and external byte length. The first three profiles use $n = 1024$, while MAMBA-NIKE-384 and MAMBA-NIKE-512 use $n = 2048$; the centered binomial parameters η_s, η_r and the compression widths $(t_{\text{pk}}, t_u, t_v)$ are tuned per profile. The MAMBA-NIKE-384 and MAMBA-NIKE-512 security rows record the current $n = 2048$

Table 4: KEX byte sizes for the submitted profiles under p-packed transmitted labels.

Instance	pk	sk _{API}	M ₁	ss
MAMBA-NIKE-128	1184	3232	1568	32
MAMBA-NIKE-192	1312	3360	1568	32
MAMBA-NIKE-256	1312	3360	1568	32
MAMBA-NIKE-384	2848	6944	3360	48
MAMBA-NIKE-512	2848	6944	3360	64

estimator outputs stored under `estimator/results/`.

The ring dimension n is the primary security lever: it determines the lattice dimension seen by the RLWE security reduction [8], and increasing n raises the attack cost at the expense of larger key, message, and RAM sizes. The compression widths (t_{pk}, t_u) trade bandwidth against quantization noise—smaller values reduce transmitted label sizes while larger values improve key-acceptance probability at higher communication cost—and the local reconciliation width t_v controls the initiator’s local quantization before helper generation but is not transmitted. The modulus $q = 2^{13}$ is fixed across all profiles; the centered binomial parameters (η_s, η_r) are selected per security target (Table 3). The full design rationale for the power-of-two modulus and the D_4 reconciliation layer is given in Section 1.1.

3.2 Space Overheads and External Byte Lengths

The NGCC KEX implementation separates public-key distribution from the one online pass. A public key contains one p-packed public-key label and one 32-byte seed. The online pass message contains the dither seed μ , one p-packed response label, and the D_4 helper vector. The helper packs four two-bit values into one byte. The submitted implementation consumes $\kappa = n/4$ reconciliation blocks, so the raw reconciliation string has length κ bits and the packed helper vector has length $n/4$ bytes in the current API profile. For a profile of dimension n , the external byte counts are

$$|\text{pk}| = 32 + \frac{nt_{\text{pk}}}{8}, \quad |\text{sk}|_{\text{API}} = 2n + |\text{pk}|$$

and

$$|M_1| = 32 + \frac{nt_u}{8} + \frac{n}{4}, \quad |\text{ss}| = \begin{cases} 32, & \text{if } n = 1024, \\ \text{NIKE_SECURITY_BITS}/8, & \text{if } n = 2048. \end{cases}$$

The core secret polynomial occupies $2n$ bytes; the NGCC API secret-key object additionally caches the corresponding public key, giving the $|\text{sk}|_{\text{API}}$ values in Table 4. The large transmitted objects are the public key polynomial and the online response polynomial. The polynomial v is not transmitted and is tuned for reliability rather than bandwidth. The helper vector is inherited from the D_4 reconciliation layer; changing it would define a different reconciliation mechanism and therefore a different KAT format.

Remark 3.1 (Consistency of Parameters). *All tables in this specification use **unified parameter values**: MAMBA-NIKE-128, MAMBA-NIKE-192, and MAMBA-NIKE-256 use $n = 1024$; MAMBA-NIKE-384 and MAMBA-NIKE-512 use $n = 2048$; all profiles use power-of-two $q = 2^{13}$; (η_s, η_r) are chosen per security level as listed in Table 3. This supersedes any parameter values appearing in earlier versions of this document.*

3.3 Correctness and Security Summary

The compression widths $(t_{\text{pk}}, t_u, t_v)$ are security-critical constants. Changing these widths changes the induced quantization-error distributions and therefore changes both the

security-estimation inputs and the key-acceptance probability. The public-key and pass-message components determine the two exposed split RLWQ $-\mathbb{Z}$ instances, while the local v component enters the correctness analysis through the agreement error

$$\Delta = e_{\text{pk}}r - e_us + e_v.$$

The security columns in Table 3 record the tuned estimator outputs used for parameter selection. The concrete security estimates and cryptanalytic discussion are given in Section 6; the passive-security proof is given in Appendix B and the exact key-acceptance formulation is given in Appendix C. The repository tool `compute_rejection.py` constructs the independent-coefficient/block-model FFT estimate, but the current double-precision implementation truncates probabilities at 10^{-13} . The implementation evidence retained in this document is the observed 5,000/5,000 agreement record for each profile.

4 Performance Evaluation

This section evaluates the computational efficiency and communication bandwidth of the MAMBA-NIKE Reference and Optimized implementations on x86-64 architecture. The submitted x86 package contains these two implementation tracks.

4.1 Experimental Setup and Platforms

The measurements reported below compare the portable Reference implementation and the AVX2 Optimized implementation under the same x86-64 benchmark environment. The hardware environment, toolchain, and build modes are summarized in Table 5.

Table 5: x86-64 benchmarking environment for the submitted implementations.

Component	x86-64 Reference and Optimized builds
Architecture	x86-64 host with AVX2 support
Environment	Linux/WSL x86-64
Compiler	GCC-compatible C99 toolchain
Reference flags	<code>-std=c99 -O3 -Wall -Wextra -fomit-frame-pointer -DKAT_BUILD</code>
Optimized flags	Reference flags plus <code>-mavx2 -march=native</code>
Timing source	RDTSC cycle counter
Statistics	Arithmetic mean of 1,000 paired timing executions; separate conformance testing uses 5,000 exchanges per profile
Reported unit	thousands of cycles (kCy)

4.2 Implementation Variants

The submission package contains two implementation tracks: a portable C Reference implementation and an x86-64 AVX2 Optimized implementation. Both tracks share the same mathematical conformance target, and the Optimized output must match the canonical Reference KATs byte for byte.

Reference implementation. The reference implementation is portable C and is written for clarity, conformance testing, and KAT generation. It uses only the coefficient-domain Toom–Cook-4 multiplication path, contains no AVX2 intrinsics or AVX2-specific compiler flags, and does not use a prime-field NTT representation.

Optimized implementation. The optimized implementation targets x86-64 platforms with AVX2 vector instructions. For every protocol multiplication in which one operand is a small centered-binomial polynomial, it uses a fixed-iteration, constant-address AVX2 multiplication routine with 16-bit lanes; the portable Toom–Cook-4 routine remains the non-AVX2 fallback. The optimized path is required to produce byte-for-byte identical outputs to the reference KEX KATs.

4.3 x86 Implementation Results

Tables 6 and 7 report paired reference and optimized measurements obtained on the same x86-64 AVX2 host using RDTSC. The reference build uses `-O3 -fomit-frame-pointer`; the optimized build additionally uses `-mavx2 -march=native`. Values are arithmetic means over 1,000 executions and are reported in thousands of cycles (kCy).

Table 6: x86-64 reference implementation performance (portable C Toom–Cook-4, RDTSC kCy; mean of 1,000 executions).

Instance	n	q	$ \text{pk} $	$ M_1 $	KeyGen	Pass1	Derive
MAMBA-NIKE-128	1024	2^{13}	1184	1568	1056	2065	1005
MAMBA-NIKE-192	1024	2^{13}	1312	1568	1058	2066	1007
MAMBA-NIKE-256	1024	2^{13}	1312	1568	1071	2083	1011
MAMBA-NIKE-384	2048	2^{13}	2848	3360	2140	4219	2085
MAMBA-NIKE-512	2048	2^{13}	2848	3360	2139	4212	2069

Table 7: x86-64 AVX2 optimized implementation performance (small-CBD polynomial multiplication, RDTSC kCy; mean of 1,000 executions). The parameter column lists $(\eta_s, \eta_r; t_{\text{pk}}, t_u, t_v)$ and makes the profile-specific differences explicit.

Instance	Parameters	n	q	$ \text{pk} $	$ M_1 $	KeyGen	Pass1	Derive
MAMBA-NIKE-128	(2, 2; 9, 10, 6)	1024	2^{13}	1184	1568	151	293	113
MAMBA-NIKE-192	(3, 3; 10, 10, 6)	1024	2^{13}	1312	1568	154	270	115
MAMBA-NIKE-256	(7, 7; 10, 10, 6)	1024	2^{13}	1312	1568	177	325	115
MAMBA-NIKE-384	(2, 2; 11, 11, 6)	2048	2^{13}	2848	3360	349	683	322
MAMBA-NIKE-512	(5, 5; 11, 11, 6)	2048	2^{13}	2848	3360	343	690	317

The x86-64 reference implementation remains a strictly portable coefficient-domain implementation: every ring product is computed by Toom–Cook-4, and neither the source files nor the build flags use AVX2. The optimized implementation exploits the fact that each protocol product has one small centered-binomial operand. It replaces the corresponding generic convolution calls with fixed-iteration AVX2 shift-and-accumulate multiplication, uses independent 16-bit SIMD lanes, permits natural reduction modulo 2^{16} during accumulation, and applies the final $(q - 1)$ mask for reduction modulo $q = 2^{13}$. Across the five parameter sets, the measured end-to-end speedups are 6.05 – $6.99\times$ for KeyGen, 6.10 – $7.65\times$ for Pass1, and 6.48 – $8.89\times$ for Derive. The optimized implementation remains byte-for-byte KAT-compatible with the Toom–Cook reference implementation.

4.4 Comparison to NewHope KE

Table 8 consolidates the main operational parameters, byte-level sizes, security estimates, and efficiency records of MAMBA-NIKE against NewHope KE, grouped by target classical security level. The comparison highlights the key structural divergences: MAMBA-NIKE uses power-of-two arithmetic and public dithered quantization (RLWQ – $\mathbb{Z}$ transport), whereas NewHope uses prime-field NTT and explicit RLWE error sampling.

Table 8: Comparison of MAMBA-NIKE with NewHope KE grouped by target security levels. Type: underlying problem family. n : ring dimension; q : arithmetic modulus; η : centered binomial parameter. Sizes in bytes: $|\text{pk}|$ public key, $|M_1|$ one-pass message, $|\text{sk}|$ secret key, and $|\text{ss}|$ shared secret. For MAMBA-NIKE, $|\text{sk}|$ is the NGCC API secret-key object `Encode16(secret polynomial) || own public key`. MATZOV C = $\min(\text{usvp}, \text{bdd})$ under MATZOV progressive BKZ; MATZOV Q = 0.292β (classical Core-SVP, β from MATZOV primal). Efficiency: x86 Ref and AVX2 numbers are paired RDTSC kCy means over 1,000 executions on the same x86-64 host. The Reference implementation uses portable-C Toom–Cook-4 only; the NGCC Optimized implementation uses the dedicated small-CBD AVX2 polynomial multiplier and is compiled with `-mavx2 -march=native`. NewHope AVX2 timings are from the USENIX Security 2016 paper [10] (Intel Haswell i7-4770K). KG = KeyGen, P1 = Pass1, Der = Derive.

Scheme	Type	n	q	η	Sizes (bytes)				Security (bits)		Efficiency (kCy)					
					pk	$ M_1 $	sk	ss	MATZOV C	MATZOV Q	x86 Ref			AVX2		
											KG	P1	Der	KG	P1	Der
128-bit Classical Security																
MAMBA-NIKE-128	RLWQ-Z	1024	2^{13}	2	1184	1568	3232	32	272.1	251.1	1056	2065	1005	151	293	113
NewHope-512 [10]	RLWE	512	12289	16	928	1024	896	32	145.7	119.4	—	—	—	86	105	24
192-bit Classical Security																
MAMBA-NIKE-192	RLWQ-Z	1024	2^{13}	3	1312	1568	3360	32	279.3	258.4	1058	2066	1007	154	270	115
NewHope-1024 [10]	RLWE	1024	12289	16	1824	2048	1792	32	297.3	277.1	—	—	—	164	195	46
256-bit Classical Security																
MAMBA-NIKE-256	RLWQ-Z	1024	2^{13}	7	1312	1568	3360	32	294.3	273.9	1071	2083	1011	177	325	115
NewHope-1024 [10]	RLWE	1024	12289	16	1824	2048	1792	32	297.3	277.1	—	—	—	164	195	46
384-bit Classical Security																
MAMBA-NIKE-384	RLWQ-Z	2048	2^{13}	2	2848	3360	6944	48	538.6	512.5	2140	4219	2085	349	683	322
512-bit Classical Security																
MAMBA-NIKE-512	RLWQ-Z	2048	2^{13}	5	2848	3360	6944	64	590.2	512.5	2139	4212	2069	343	690	317

As shown in Table 8, MAMBA-NIKE is a non-interactive single-pass key exchange (one online message) while NewHope requires two passes (interactive). At the 128-bit level, the MAMBA-NIKE-128 profile achieves p-packed bandwidth of 1184 B (pk) and 1568 B (M_1), while the NewHope-512 row uses 928 B and 1024 B respectively; NewHope-1024 is retained for the 192- and 256-bit comparison rows. The portable MAMBA-NIKE reference path retains coefficient-domain Toom–Cook-4 for auditability, while the AVX2 path substantially reduces its end-to-end cost through the dedicated small-CBD polynomial multiplier. The 192-bit and 256-bit levels use the same $n = 1024$ and $t_u = 10$ as each other, differing only in η ; MAMBA-NIKE-192 ($\eta = 3$) and MAMBA-NIKE-256 ($\eta = 7$) thus achieve identical byte lengths (1312 B pk, 1568 B M_1) with different concrete security margins. For the 384-bit and 512-bit levels, MAMBA-NIKE uses $n = 2048$, $t_{\text{pk}} = t_u = 11$, with $\eta = 2$ and $\eta = 5$ respectively, yielding identical byte lengths (2848 B pk, 3360 B M_1).

Paired x86-64 optimization verification (2026-06-30). All five profiles were compiled and tested in both forms: the portable reference implementation with Toom–Cook-4 only and the x86-64 optimized implementation with the dedicated AVX2 small-CBD multiplier. The paired RDTSC measurements (reference/AVX2, kCy) were: MAMBA-NIKE-128 (KG=1056/151, P1=2065/293, Der=1005/113), MAMBA-NIKE-192 (KG=1058/154, P1=2066/270, Der=1007/115), MAMBA-NIKE-256 (KG=1071/177, P1=2083/325, Der=1011/115), MAMBA-NIKE-384 (KG=2140/349, P1=4219/683, Der=2085/322), and MAMBA-NIKE-512 (KG=2139/343, P1=4212/690, Der=2069/317). For conformance, each optimized profile completed 5,000/5,000 successful key agreements with zero observed failures, and its raw KAT output matched the corresponding canonical reference KAT byte for byte.

5 Known Answer Tests

The official NGCC known-answer tests for MAMBA-NIKE are generated under `API_PKC/KAT/` in the submission package and correspond to the formal KEX test-vector material required by the NGCC `API_PKC` interface. This directory is the generation directory used by the submitted scripts; the final submitted test-vector files are mirrored under `Test_Vectors/` according to the NGCC submission layout. The directory contains one pair of `.req/.rsp` files for each submitted parameter set. These files are generated by the unmodified official `KAT_KEX.c` copied into the Reference Implementation instance directory together with the support files `drng.c`, `drng.h`, `auxfunc.c`, and `auxfunc.h`.

For each parameter set, running `bash generate_all_kats.sh` from `API_PKC/` compiles the corresponding self-contained Reference instance using its profile-specific `params.h`, executes the official KEX KAT generator, and uses `parse_kat.py` to produce the formal `.req` and `.rsp` files. The Reference output is the formal KAT value; each Optimized output is checked against it byte for byte.

Table 9: Official NGCC `API_PKC` KAT files for the MAMBA-NIKE parameter sets.

Scheme	pk	sk	M_1	ss	KAT file
MAMBA-NIKE-128	1184	3232	1568	32	<code>API_PKC/KAT/MAMBA-NIKE-128/PQCkexKAT_2752.rsp</code>
MAMBA-NIKE-192	1312	3360	1568	32	<code>API_PKC/KAT/MAMBA-NIKE-192/PQCkexKAT_2880.rsp</code>
MAMBA-NIKE-256	1312	3360	1568	32	<code>API_PKC/KAT/MAMBA-NIKE-256/PQCkexKAT_2880.rsp</code>
MAMBA-NIKE-384	2848	6944	3360	48	<code>API_PKC/KAT/MAMBA-NIKE-384/PQCkexKAT_6208.rsp</code>
MAMBA-NIKE-512	2848	6944	3360	64	<code>API_PKC/KAT/MAMBA-NIKE-512/PQCkexKAT_6208.rsp</code>

Each KAT file consists of a sequence of test records. A record contains the deterministic test seed, the generated initiator and responder public keys, secret keys, state buffers, the one-pass message, and the shared secret, together with their byte lengths. The fields are written as `Count`, `Seed_Len`, `Seed`, `Pass_Num`, `PKa_Len`, `PKa`, `SKa_Len`, `SKa`, `Init_Sta_Len`, `Init_Sta`, `PKb_Len`, `PKb`, `SKb_Len`, `SKb`, `Init_Stb_Len`, `Init_Stb`, `Pass1_Sta_Len`, `Pass1_Sta`, `M1_Len`, `M1`, `SS_Len`, and `SS`. The byte lengths in the KAT records match the external API lengths reported in Table 9 and Table 4. All byte strings are uppercase hexadecimal without separators.

6 Security Strength

This section proves the passive security of the submitted MAMBA-NIKE key-exchange primitive directly, without first embedding the construction into a PKE or KEM abstraction. This is the appropriate proof route for MAMBA-NIKE because the online message

$$M_1 = (\mu, u, h)$$

is not a ciphertext encrypting a plaintext. Instead, it consists of a public dither seed, a split quantization label, and a public D_4 reconciliation helper. The responder does not decrypt; it reconstructs a close algebraic value and applies reconciliation.

The proof target is therefore passive real-or-random session-key indistinguishability for the one-pass transcript. Authentication, active attacks, replay protection, identity binding, UKS/KCI resistance, channel binding, and decapsulation-oracle security are outside the submitted standalone KE interface. The key-acceptance probability is reported separately because rare reconciliation rejections may become security-relevant if an implementation or an external protocol leaks rejection information [20, 21].

6.1 Security Definitions and Claims

The submitted primitive is a standalone one-pass key-exchange mechanism after public-key distribution. The public interface consists of key generation, initiator-side generation of the one online message, and responder-side shared-secret derivation:

$$\begin{aligned} (\text{pk}_B, \text{sk}_B) &\leftarrow \text{MAMBA-NIKE.KeyGen}, \\ (M_1, K_A) &\leftarrow \text{MAMBA-NIKE.Pass1}(\text{pk}_B), \\ K_B &\leftarrow \text{MAMBA-NIKE.Derive}(\text{sk}_B, \text{pk}_B, M_1). \end{aligned}$$

The transcript observed by a passive adversary is

$$T = (\text{pk}_B, M_1) = ((\rho, b), (\mu, u, h)).$$

Let ℓ_{ss} denote the declared shared-secret length in bits for the selected profile.

Definition 6.1 (Passive real-or-random KE advantage). *Let $\mathcal{A}$ be a passive adversary against the standalone MAMBA-NIKE key-exchange experiment. The challenger samples $(\text{pk}_B, \text{sk}_B) \leftarrow \text{MAMBA-NIKE.KeyGen}$ and gives pk_B to $\mathcal{A}$. It then samples a challenge bit $c \leftarrow \mathcal{U}(\{0, 1\})$, runs*

$$(M_1, K_A) \leftarrow \text{MAMBA-NIKE.Pass1}(\text{pk}_B),$$

samples $K_1 \leftarrow \mathcal{U}(\{0, 1\}^{\ell_{\text{ss}}})$, sets $K_0 = K_A$, and gives (M_1, K_c) to $\mathcal{A}$. The adversary outputs c' , and its advantage is

$$\text{Adv}_{\text{MAMBA-NIKE}}^{\text{ror}}(\mathcal{A}) = \left| \Pr[c' = c] - \frac{1}{2} \right|.$$

The experiment exposes no decryption, decapsulation, validation, active-session, reveal, corrupt, or test-session oracle. It captures passive session-key indistinguishability for the one-pass KE primitive only.

Decisional RLWE advantage. Let χ_s and χ_e be distributions over R . For a distinguisher $\mathcal{B}$, define

$$\begin{aligned} \text{Adv}_{\text{RLWE}}^{\text{dec}}(\mathcal{B}; q, \chi_s, \chi_e) &\triangleq \left| \Pr[\mathcal{B}(a, y) = 1 : a \leftarrow \mathcal{U}(R_q), s \leftarrow \chi_s, e \leftarrow \chi_e, y = as + e] \right. \\ &\quad \left. - \Pr[\mathcal{B}(a, y) = 1 : a \leftarrow \mathcal{U}(R_q), y \leftarrow \mathcal{U}(R_q)] \right|. \end{aligned}$$

For multi-sample variants, the same secret is used across the samples and all public ring elements and errors are sampled independently.

Decisional RLWQ advantage. Let $q > p$, let $\Delta = q/p$, let $\Lambda = \Delta\mathbb{Z}^n$, and let

$$\mathcal{D}_\Lambda = \{d \in R : \tau(d)_i \in \{0, \dots, \Delta - 1\} \text{ for all } i\}.$$

For a distinguisher $\mathcal{B}$, define

$$\text{Adv}_{\text{RLWQ}}^{\text{dec}}(\mathcal{B}; q, p, \chi_s) = |\Pr[\mathcal{B}(A_{\text{real}}) = 1] - \Pr[\mathcal{B}(A_{\text{rand}}) = 1]|.$$

Here $A_{\text{real}} = (a, c)$ with $a \leftarrow \mathcal{U}(R_q)$, $s \leftarrow \chi_s$, $d \leftarrow \mathcal{U}(\mathcal{D}_\Lambda)$, and $c = \mathcal{Q}_{\Lambda, d}^\diamond(as) - d$. The random distribution is $A_{\text{rand}} = (a, c)$ with $a, c \leftarrow \mathcal{U}(R_q)$ independently. This is the normal-form distinguishing advantage: the dither has been subtracted and the exposed value is interpreted as an RLWE-type sample with input-independent quantization error.

Decisional split-RLWQ advantage. The actual transcript exposes the split label and the public dither seed rather than only the normal-form value. For a distinguisher $\mathcal{B}$, define

$$\mathbf{Adv}_{\text{sRLWQ}}^{\text{dec}}(\mathcal{B}; q, p, \chi_s) = |\Pr[\mathcal{B}(S_{\text{real}}) = 1] - \Pr[\mathcal{B}(S_{\text{rand}}) = 1]|.$$

Here $S_{\text{real}} = (a, d, c)$ with $a \leftarrow \mathcal{U}(R_q)$, $s \leftarrow \chi_s$, $d \leftarrow \mathcal{U}(\mathcal{D}_\Lambda)$, and $c = \mathcal{Q}_{\Lambda, d}^\diamond(as)$. The random distribution is $S_{\text{rand}} = (a, d, c)$ with $a \leftarrow \mathcal{U}(R_q)$, $d \leftarrow \mathcal{U}(\mathcal{D}_\Lambda)$, and $c \leftarrow \mathcal{U}(R_p)$ independently. The split definition is the one matched to the transmitted public key and one-pass response. The normal-form definition is used for concrete estimator compatibility after reconstructing $\Delta c - d \pmod q$.

Definition 6.2 (Reconciliation leakage and key acceptance). *Let ν be the κ -bit raw reconciliation string produced by $\text{HelpRec}(\hat{v})$, where $\kappa = n/4$ for the selected profile, and let h be the corresponding public helper. For a transcript $T = (\text{pk}_B, M_1)$, define ε_{rec} as an upper bound on the statistical distance between ν conditioned on T and a uniform κ -bit string. Let*

$$\rho_{\text{key}} = \Pr[\text{Rec}(w, h) \neq \text{Rec}(\hat{v}, h)], \quad P_{\text{acc}} = 1 - \rho_{\text{key}}.$$

The quantity ρ_{key} is the key-rejection probability, and P_{acc} is the key-acceptance probability. Appendix C.1 gives the block-wise computable upper bound used for correctness accounting.

6.2 Theoretical Security

Claim 6.1 (Public-key split-sample pseudorandomness). *For each submitted parameter set, the responder public key*

$$\text{pk}_B = (\rho, b)$$

is computationally indistinguishable from a public seed together with a uniform split label under the decisional split-RLWQ assumption for the public-key layer.

Claim 6.2 (One-pass response split-sample pseudorandomness). *For each submitted parameter set, the response component*

$$u = \mathcal{Q}_{\Lambda_u, d_u}^\diamond(ar)$$

in the one-pass message is computationally indistinguishable from a uniform response split label under the decisional split-RLWQ assumption for the one-pass response layer.

Claim 6.3 (Reconciliation leakage and key acceptance). *Conditioned on the public transcript $T = (\text{pk}_B, M_1)$, the public helper h leaks about the raw reconciliation string only through the statistical term ε_{rec} in Definition 6.2. The responder accepts the same raw key as the initiator except with key-rejection probability ρ_{key} . Both quantities are controlled by the D_4 reconciliation analysis in Appendix C.1.*

Claim 6.4 (Passive real-or-random KE security). *The submitted MAMBA-NIKE primitive targets passive real-or-random KE security under the decisional split-RLWQ assumptions for the public-key and one-pass response layers, the reconciliation-leakage bound, the key-rejection bound, and the transcript-bound SHAKE256 KDF step [2, 8, 11, 27].*

Theorem 6.1 (Direct passive real-or-random security). *For every probabilistic polynomial-time passive adversary $\mathcal{A}$ against the one-pass KE experiment, there exist probabilistic polynomial-time distinguishers $\mathcal{B}_{\text{pk}}$ and $\mathcal{B}_u$ such that*

$$\mathbf{Adv}_{\text{MAMBA-NIKE}}^{\text{ror}}(\mathcal{A}) \leq \mathbf{Adv}_{\text{sRLWQ}}^{\text{dec}}(\mathcal{B}_{\text{pk}}; q, p_{\text{pk}}, \mathcal{B}_{\eta_s}) + \mathbf{Adv}_{\text{sRLWQ}}^{\text{dec}}(\mathcal{B}_u; q, p_u, \mathcal{B}_{\eta_r}) + \varepsilon_{\text{rec}} + \rho_{\text{key}}.$$

The last term is the correctness loss for experiments that require the responder key to match the initiator key; for pure initiator-key indistinguishability it can be omitted.

Table 10: Consolidated concrete security estimates for the MAMBA-NIKE parameter sets. primal + BKW use the MATZOV cost model; dual + d-hyb use the ADPS16 cost model (MATZOV dual unavailable in this estimator version). Errors are modelled as zero-mean Uniform($-\Delta/2, \Delta/2$), same variance as the exact Uniform($-\Delta/2 + 1, \Delta/2$).

Scheme	Target	primal	dual	d-hyb	BKW	MATZOV C	MATZOV Q
MAMBA-NIKE-128	128	272.1	265.7	243.1	315.1	272.1	251.1
MAMBA-NIKE-192	192	279.3	273.3	251.5	331.6	279.3	258.4
MAMBA-NIKE-256	256	294.3	289.4	283.0	367.3	294.3	273.9
MAMBA-NIKE-384	384	538.6	747.9	495.0	–	538.6	512.5
MAMBA-NIKE-512	512	590.2	1119.3	571.5	–	590.2	512.5

The proof is a standard hybrid over the public transcript. First replace the responder public-key split sample by a uniform split sample, then replace the one-pass response split sample by a uniform split sample, and finally replace the raw reconciliation string by a uniform κ -bit string using the reconciliation-leakage bound. The transcript-bound KDF then produces a uniform shared secret of the declared profile length from the passive adversary’s view. Appendix B gives the full game sequence and Appendix C gives the correctness accounting.

6.3 Classical and Quantum Security Levels

Table 10 summarizes the concrete security records used for the submitted parameter sets. The classical columns exceed the nominal target for all five profiles, and the quantum column remains above the corresponding target in the current records. These numbers are parameter-selection evidence for the exposed split-RLWQ layers; they do not expand the security claim beyond passive real-or-random KE security.

6.4 Side-Channel Security Considerations

Side-channel and fault attacks are outside the passive real-or-random experiment and outside the lattice-estimator model. The submitted arithmetic is nevertheless structured to support constant-time implementations: moduli are powers of two, reductions use masking and fixed-width arithmetic, loops are determined by public parameter sizes, and external buffer lengths are fixed by the selected profile. The statements below are implementation requirements and design rationale, not a claim of leakage resilience without platform-specific testing.

6.4.1 Timing and Microarchitectural Attacks

The core operations should avoid secret-dependent branches and secret-dependent memory addresses. Polynomial multiplication uses public loop bounds, coefficient reduction is mask-based, and quantization/reconstruction uses additions, shifts, and fixed public moduli. The D_4 reconciliation routines should be implemented with branchless table-free arithmetic or with tables indexed only by public block positions. These choices are intended to reduce timing, cache-timing, and microarchitectural leakage surfaces such as Prime+Probe and Flush+Reload [24, 25].

6.4.2 Power and Electromagnetic Leakage

The main secret-dependent values are the long-term secret s , the ephemeral secret r , and products involving these polynomials. Physical leakage resistance therefore requires masking,

hiding, or leakage-tested platform code for deployments with power or electromagnetic adversaries. The public dither values and public seeds do not themselves require protection as secrets, but intermediate products involving s or r must be treated as sensitive until they are overwritten. The reference implementation is a conformance target; protected embedded implementations should add masking and validation appropriate to the target device [22, 26].

6.4.3 Fault and State-Handling Attacks

Fault attacks primarily target secret-key use, one-pass message parsing, helper processing, and final shared-secret state. Implementations should validate all public buffer lengths, reject malformed encodings before arithmetic use, keep pending session state in fixed-size buffers, and clear ephemeral secrets and raw reconciliation strings after key derivation. Because MAMBA-NIKE is unauthenticated, replay handling, identity binding, and channel binding must be provided by the higher-level protocol that deploys the primitive.

7 Advantages and Limitations

This section provides an honest assessment of the MAMBA-NIKE scheme, highlighting its strengths, limitations, and areas requiring further investigation during the NGCC review process.

7.1 Advantages

- (1) **Compactness via Quantization Compression.** The RLWQ $-\mathbb{Z}$ framework enables profile-specific compression through power-of-two quantization. As shown in Table 4, the default MAMBA-NIKE-128 KEX profile uses a 1568-byte public key and a 1824-byte one-pass message under p-packed transmitted labels.
- (2) **Input-Independent Quantization Error.** The dithered quantization mechanism produces errors that are statistically independent of the input values (Theorem A.1). This property fundamentally simplifies the security analysis: the key-rejection probability can be bounded without conditioning on specific secret or error values, yielding cleaner and more rigorous security proofs compared to schemes where failure events depend on the secret key structure.
- (3) **Power-of-Two Arithmetic.** All submitted profiles use $q = 2^\ell$. This makes coefficient reduction a branchless mask operation and removes the need for prime-field roots of unity. Polynomial multiplication is implemented by coefficient-domain negacyclic Toom-Cook convolution.
- (4) **Unified Code Architecture.** All five security profiles (MAMBA-NIKE-128 through MAMBA-NIKE-512) share a *single unified codebase*. The profile differences are compile-time constants such as n , q , p_{pk} , p_u , and p_v . This architectural uniformity significantly reduces implementation effort, maintenance burden, and code audit complexity.
- (5) **Explicit and Transparent Error Analysis.** The shared-key computation involves an explicit error term:

$$\Delta = e_{\text{pk}} \cdot r - e_u \cdot s + e_v, \quad (1)$$

where e_{pk} is the long-term key quantization error, e_u is the ephemeral quantization error, and e_v is the reconciliation error. This closed-form expression enables precise key-acceptance estimation and direct connection to well-studied RLWE error distributions.

- (6) **NGCC KEX Compatibility.** MAMBA-NIKE is implemented against the NGCC KEX interface. The API reports one online pass, exposes deterministic byte lengths, and generates KEX KAT files in the required response format.
- (7) **Operational Simplicity.** The design uses a small set of uniform building blocks: power-of-two masking reduction, coefficient-domain Toom–Cook convolution, profile-specific centered-binomial sampling, public dither generation, and NewHope-style D_4 reconciliation. This keeps the specification concise and reduces implementation-audit complexity.
- (8) **Flexible and Scalable Parameterization.** The five submitted profiles are obtained by varying the ring dimension and compression widths while keeping the same algebraic structure, API flow, and implementation architecture. This supports security-performance trade-offs from 128-bit to 512-bit classical security without changing the protocol syntax.
- (9) **Two-Track Implementation Support.** The Reference implementation is portable C and the Optimized implementation targets mainstream x86-64 platforms with AVX2. Both tracks preserve the same external byte layout and KAT behavior.

7.2 Limitations

- (1) **Communication Overhead at High Security Levels.** Higher security profiles increase the polynomial dimension and therefore public-key and pass-message sizes. In the current parameter record, MAMBA-NIKE-384 and MAMBA-NIKE-512 use $n = 2048$. This is the main cost of the higher estimator margins.
- (2) **Dependence on the RLWE Hardness Assumption.** The security of MAMBA-NIKE rests entirely on the hardness of the RLWE (and $RLWQ - \mathbb{Z}$) problem. Should a breakthrough algorithm for RLWE be discovered—whether classical or quantum—MAMBA-NIKE would become insecure. This is a shared limitation of all lattice-based cryptographic schemes. We mitigate this risk by selecting conservative parameters with substantial security margins (see Table 10).
- (3) **Non-Unit Key-Acceptance Probability.** Like practical reconciliation-based lattice schemes, MAMBA-NIKE has a key-acceptance probability below one in the mathematical model. Appendix C gives the exact full-key formulation used to bound the key-rejection probability, and Table 11 records successful key-agreement tests for all submitted profiles.
- (4) **Hardware Implementation Scope.** The first-round material emphasizes reference C, optimized x86-64 software, and embedded-software portability. FPGA and ASIC performance are outside the claimed performance figures in this document, although the arithmetic layer is structured to support later hardware evaluation.
- (5) **Security Margin Analysis for 384/512-Bit Profiles.** MAMBA-NIKE-384 and MAMBA-NIKE-512 use larger dimensions than the lower profiles and meet the recorded estimator targets. Their main tradeoff is implementation cost: the larger dimensions increase public-key size, one-pass message size, and RAM usage.

MAMBA-NIKE presents a carefully engineered non-interactive key exchange mechanism that leverages the $RLWQ - \mathbb{Z}$ framework to achieve competitive performance with rigorous security foundations. The scheme’s unified architecture, explicit error analysis, and compatibility with the broader MAMBA-NIKE suite make it a strong candidate for standardization within the NGCC ecosystem.

The current submission includes a complete KE specification, reference implementation material, optimized implementation material, KAT-oriented conformance records, concrete security estimates, and explicit discussion of limitations. We believe that MAMBA-NIKE offers a balanced combination of compactness, security, and implementation efficiency, and we welcome the evaluation panel’s feedback to further strengthen this proposal.

8 Implementation Specification

This section specifies the engineering conventions required for interoperable MAMBA-NIKE implementations. The mathematical algorithms are specified in Section 2; the corresponding notation, byte encodings, and the full NGCC KEX API are specified in Section 8.1. This section fixes the implementation boundary, byte layouts, integer representation, expansion rules, sampling order, quantization rules, and constant-time execution requirements.

The submitted `API_PKC` package is the formal implementation boundary. It contains two implementation tracks, each with five profile-specific instances: Reference and Optimized. The Reference implementation is the portability baseline. Optimized implementations may use different internal layouts, vectorized arithmetic, or precomputation, but they must produce identical public keys, one-pass messages, shared secrets, and Known Answer Test outputs for the same inputs.

All external objects are byte strings. Polynomial coefficients are serialized in increasing coefficient order. Unless explicitly stated otherwise, all profile-dependent byte lengths are the declared API lengths returned by the corresponding `kex_get_*` functions and are listed in Table 4.

8.1 External KEX API and Implementation Boundary

For each submitted profile, the formal implementation exposes the NGCC KEX interface through the functions listed in below. The API reports one online pass via `kex_get_passes_num()`, which returns 1 for all profiles. The length-query functions `kex_get_pk_len_bytes`, `kex_get_sk_len_bytes`, `kex_get_sta_len_bytes`, `kex_get_stb_len_bytes`, `kex_get_ss_len_bytes`, and `kex_get_total_msg_len_bytes` return the profile-dependent byte lengths described below.

The submitted implementation is organized under the NGCC `API_PKC` layout. The Reference and Optimized tracks each contain five self-contained profile directories. Each directory carries a profile-specific `params.h`; the parameter records are validated against the immutable `parameters.json` manifest. The files `KAT_KEX.c`, `drng.c/drng.h`, and `auxfunc.c/auxfunc.h` are the official NGCC support files and are used without modification.

Internal byte-length constants. Each profile fixes a small set of compile-time constants. The submitted profiles use $n \in \{1024, 2048\}$, $q = 2^{13} = 8192$, and profile-specific centered-binomial parameters listed in Table 3. The key internal constants are: `PARAM_N` (ring dimension), `PARAM_K` (profile-specific centered binomial parameter), `PARAM_Q` (arithmetic modulus, 2^{13}), and `LOG2Q` ($\log_2 q = 13$). The transmitted byte-length constants follow the p-packed accounting described in Section 3.1:

$$\text{SEEDBYTES} = 32, \quad \text{RECBYTES} = n/4, \quad \text{POLY_BYTES} = 2n,$$

$$\text{SENDA_BYTES} = 32 + nt_{\text{pk}}/8, \quad \text{SENDB_BYTES} = 32 + nt_u/8 + n/4.$$

The KEX API types map to these constants as follows:

$$\text{PK_BYTES} = \text{SENDA_BYTES}, \quad \text{M1_BYTES} = \text{SENDB_BYTES},$$

$$\text{SK_BYTES} = \text{POLY_BYTES} + \text{SENDA_BYTES}, \quad \text{SS_BYTES} = \text{NIKE_SSBYTES}.$$

The secret key (**sk**) concatenates the serialized secret polynomial s ($2n$ bytes) with the corresponding public key (**SENDA_BYTES** bytes); this is required for transcript-bound key derivation in **nike_shareda**. The initiator state (**sta**) stores the pre-computed shared secret (**SS_BYTES** bytes) during pass generation and is retrieved by the initiator-side derivation function. The responder state (**stb**) is unused (0 bytes) in the one-pass NIKE flow. The shared-secret length **SS_BYTES** is profile-dependent and follows Table 4: 32 bytes for $n = 1024$ profiles, 48 bytes for MAMBA-NIKE-384, and 64 bytes for MAMBA-NIKE-512.

API behavior.

- **kex_init_a** and **kex_init_b** each generate a long-term keypair via **nike_keygen**, returning the public key and the secret polynomial concatenated with its public key. Both initialize their state buffers to zero length.
- **kex_generate_pass1_msg_a** takes the initiator secret key, the responder public key, and returns the one-pass message M_1 together with the shared secret stashed in the initiator state buffer. The implementation calls **nike_sharedb**, which samples a fresh ephemeral secret internally and produces both M_1 and the shared secret in a single call.
- **kex_derive_ss_a** retrieves the pre-computed shared secret from the initiator state buffer without re-derivation.
- **kex_derive_ss_b** deserializes the responder secret polynomial and calls **nike_shareda** to compute the shared secret from the received message M_1 .
- **kex_generate_pass2_msg_b** and **kex_generate_pass3_msg_a** are compatibility stubs that return without producing protocol messages, because **kex_get_passes_num()** returns 1.

8.2 Integer Representation

All polynomial coefficients are represented internally as unsigned 16-bit words (**uint16_t**). Since $q \leq 2^{13}$ for all submitted profiles, this representation covers all coefficient values. Reduction modulo q is implemented by masking with $q - 1$:

$$a \bmod q = a \& (q - 1).$$

No Montgomery reduction, Barrett reduction, or modular inversion is required. The centered interpretation is used only for analysis and intermediate signed reasoning. Serialized values use the canonical nonnegative representation $[0, q)$.

For each quantized component $x \in \{\text{pk}, u, v\}$, set $p_x = 2^{t_x}$, $\Delta_x = q/p_x = 2^{\log_2 q - t_x}$, and $g_x = \log_2 \Delta_x = \log_2 q - t_x$. Multiplication by Δ_x and division by Δ_x are implemented by left and right shifts, respectively. The implementation uses fixed-width integer types and fixed-latency operations throughout.

8.3 Serialized Objects

The public key consists of the p-packed split public-key label followed by the 32-byte public seed:

$$\text{Encode}(\text{pk}) = \text{Pack}_{t_{\text{pk}}}(b) \parallel \rho.$$

Here Pack_t denotes coefficient bit-packing of a split label in R_{2^t} into $nt/8$ bytes. Internally, coefficients may be represented as unsigned 16-bit words for arithmetic convenience, but

the transmitted public-key and response labels use the p-packed external representation specified in Table 4.

The one-pass message consists of the 32-byte dither seed μ , the p-packed split response label, and the packed D_4 helper:

$$\text{Encode}(M_1) = \mu \parallel \text{Pack}_{t_u}(u) \parallel h_{\text{packed}}.$$

The helper packs four two-bit values into one byte as described in Section 8.4, producing $n/4$ bytes. The core secret polynomial is serialized as $2n$ bytes using the implementation's fixed 16-bit coefficient representation; the NGCC API secret-key object appends the corresponding public key and therefore occupies $2n + |\text{pk}|$ bytes.

In summary, the external byte layout is as follows: **pk** concatenates $\text{Pack}_{t_{\text{pk}}}(b)$ with the 32-byte public seed ρ ; M_1 concatenates the 32-byte dither seed μ , $\text{Pack}_{t_u}(u)$, and the $n/4$ -byte packed helper h_{packed} ; the NGCC API **sk** concatenates the serialized secret polynomial with the corresponding public key; and **ss** is the SHAKE256 output of the profile-dependent byte length listed in Table 4.

8.4 Bit Packing and Parsing

The D_4 helper vector packs four two-bit values into each output byte. For helper polynomial $c \in R_4$ (each coefficient in $\{0, 1, 2, 3\}$), the reference implementation writes

$$h_{\text{packed}}[i] = c[4i+0] \mid (c[4i+1] \ll 2) \mid (c[4i+2] \ll 4) \mid (c[4i+3] \ll 6), \quad i = 0, \dots, \frac{n}{4} - 1.$$

The inverse unpacking is performed by the responder during helper parsing. The transmitted quantization labels b and u are coefficient-bit-packed with widths t_{pk} and t_u , respectively, by the fixed-loop `poly_pack_bits` routine; parsing uses the inverse `poly_unpack_bits` routine.

8.5 Seed Expansion and Dither Generation

The public seed ρ deterministically expands into the public polynomial $a \in R_q$ and the public-key dither $d_{\text{pk}} \in \mathcal{D}_{\text{pk}}$. The reference implementation uses SHAKE-128 for this expansion: coefficients of a are read as 16-bit little-endian words and masked modulo q ; dither coefficients are masked modulo Δ_{pk} . The dither seeds are of fixed length 32 bytes each.

The one-pass seed μ deterministically expands into the response dithers $d_u \in \mathcal{D}_u$ and $d_v \in \mathcal{D}_v$, using the same SHAKE-128 interface with distinct domain-separation labels. All expansion calls must use different domain-separation constants to prevent stream collisions across roles.

The secret seed is expanded via ChaCha20 into centered-binomial samples as described in Section 8.6. The final shared secret is derived by SHAKE256 over a fixed 16-byte domain separator, the raw reconciliation string, the responder public key, and the complete one-pass message. The SHAKE256 output length is selected by the profile.

8.6 Secret Sampling

Long-term and ephemeral secrets are sampled from profile-specific centered binomial distributions. The submitted values are $\eta = 2, 3, 7, 2, 5$ for MAMBA-NIKE-128, -192, -256, -384, and -512, respectively. The implementation expands a seed and nonce through ChaCha20 into a bit stream. For coefficient index i , it consumes the next 2η bits and computes

$$a_i = \sum_{j=0}^{\eta-1} b_{2\eta i+j}, \quad b'_i = \sum_{j=0}^{\eta-1} b_{2\eta i+\eta+j}, \quad \text{coeff}_i = (a_i - b'_i) \bmod q.$$

Thus each coefficient lies in $\{-\eta, \dots, \eta\}$ and is stored modulo q in the unsigned internal representation.

The long-term secret s and the ephemeral secret r are sampled from independent fresh 32-byte seeds. The current protocol calls use nonce value 0 within each independent stream. Coefficients are filled in increasing polynomial index order. The implementation uses a fixed number of ChaCha20 output bytes and does not perform rejection sampling.

8.7 Dithered Quantization and Compensation

Let $x \in \{\text{pk}, u, v\}$, and let t_x , Δ_x , and g_x be the corresponding compression width, quantization step, and shift value. For an input coefficient $a \in \mathbb{Z}_q$ and a dither coefficient $d \in \{0, \dots, \Delta_x - 1\}$, the transmitted split label is

$$b = ((a + d + 2^{g_x-1}) \gg g_x) \& (2^{t_x} - 1).$$

Thus $b \in \mathbb{Z}_{p_x}$ is the compact split coefficient stored in the transmitted object. The compensated normal-form coefficient, used for all arithmetic, is reconstructed as

$$\hat{a} = ((b \ll g_x) - d) \& (q - 1).$$

This rule is applied coefficient-wise to all quantized components. The serialized objects store the split labels b and u , while arithmetic uses the compensated normal forms $\hat{b}$ and $\hat{u}$. The local reconciliation component v is never transmitted; its compensated normal form $\hat{v}$ is used only for helper generation and initiator-side key derivation.

8.8 Polynomial Arithmetic

The core ring arithmetic is over $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. The reference implementation uses portable C polynomial operations in the coefficient domain. Polynomial addition computes

$$r[i] = (a[i] + b[i]) \& (q - 1), \quad i = 0, \dots, n - 1.$$

Polynomial multiplication is performed in three steps:

1. The full integer product is computed via Toom–Cook-4 multiplication [12, 13] on 64-bit signed coefficient arrays.
2. The result is folded modulo $X^n + 1$: $r[i] = \text{prod}[i] - \text{prod}[i + n]$ for $i = 0, \dots, n - 1$.
3. Coefficients are reduced modulo q by masking: $r[i] \& (q - 1)$.

The Toom–Cook-4 base case uses schoolbook multiplication when the polynomial length drops below a compile-time cutoff. This coefficient-domain convolution is used for all ring products in the Reference implementation. In AVX2 builds, the Optimized implementation redirects the four protocol products having one centered-binomial operand to `poly_mul_small()`; the generic Toom–Cook routine remains present as the non-AVX2 fallback.

The reference code retains `poly_ntt()` and `poly_invntt()` as compatibility no-ops inherited from the NewHope-style code organization. Polynomials remain in the coefficient domain throughout the protocol. The submitted Reference Implementation uses this Toom–Cook-4 path exclusively and contains no AVX2 implementation. The submitted Optimized Implementation adds a separate fixed-iteration AVX2 routine for products with one small centered-binomial operand; it falls back to Toom–Cook-4 when AVX2 is unavailable and remains subject to the KAT compatibility requirements in Section 8.11.

8.9 D_4 Reconciliation

The D_4 reconciliation layer is implemented by `helpRec` and `rec` in the reference code. It operates on

$$\kappa = n/4$$

consumed four-coefficient blocks. For each block $j \in \{0, \dots, \kappa - 1\}$, the input is

$$\text{blk}_j(x) = (x_j, x_{j+\kappa}, x_{j+2\kappa}, x_{j+3\kappa}),$$

extracted from the full n -coefficient polynomial x . The function `helpRec` computes the four two-bit helper values for each block using a randomized helper-generation rule parameterized by one ChaCha20-derived random bit per block, and packs them into an $n/4$ -byte helper vector. The function `rec` applies the received helper to recover one raw key bit per block, producing a κ -bit raw reconciliation string $\nu \in \{0, 1\}^\kappa$. Thus the helper is 256 bytes for $n = 1024$ profiles and 512 bytes for $n = 2048$ profiles.

The correctness condition for one block is

$$\|\text{blk}_j(\hat{v} - w)\|_1 < T_{D_4}, \quad T_{D_4} = \lfloor 3q/4 \rfloor - 2 = 6142,$$

where $T_{D_4} = 6142$ for $q = 2^{13}$. The initiator evaluates $\text{Rec}(\hat{v}, h)$, while the responder evaluates $\text{Rec}(w, h)$ with $w = \hat{u}s_B$. If all κ blocks satisfy the correctness condition, the two parties recover the same raw reconciliation string.

8.10 Constant-Time Requirements

All operations whose inputs depend on the secret polynomials s or r must execute without secret-dependent branches or secret-dependent memory addresses. This includes centered-binomial sampling, quantization, normal-form compensation, packing, unpacking, polynomial multiplication, addition, and reconciliation.

The power-of-two modulus enables branchless modular reduction via masking. The polynomial multiplication loop bounds are determined by the public parameter n . The ChaCha20-based noise sampler consumes a fixed number of output bytes and does not use rejection sampling. The quantization and compensation operations use only additions, shifts, and masks with public shift counts. The D_4 reconciliation routines use the same public block positions for every execution. Compatibility functions for later KEX passes must not generate additional protocol messages. This includes `kex_generate_pass2_msg_a`, `kex_derive_ss_from_pass2`, and `kex_derive_ss_from_pass3`.

8.11 Validation Requirements

A conforming implementation must satisfy the following requirements. The serialized `pk`, `sk`, `M1`, and `ss` byte strings must have exactly the declared profile lengths listed in Table 4. Serialization, deserialization, seed expansion, CBD sampling, dither generation, quantization, normal-form compensation, polynomial multiplication, helper generation, and reconciliation must be deterministic and match the Reference output for the same inputs. The Reference and Optimized instances must pass the same official `API_PKC KAT` files listed in Section 5. The Optimized implementation must preserve the same external byte layout, one-pass flow, and constant-time property as the Reference implementation. The polynomial arithmetic must produce the same coefficient values in R_q regardless of internal multiplication strategy.

A Mathematical Foundations

This section defines the mathematical objects and hardness assumptions underlying MAMBA-NIKE.

A.1 The RLWQ Problem

We recall RLWE [8] and extend LWQ [2] to the ring setting, leading to the definitions of RLWQ and Split-RLWQ below.

Definition A.1 (RLWE). *Let $q \in \mathbb{N}$, and let χ_s, χ_e be distributions over R_q . The RLWE distribution $\text{RLWE}_{q, \chi_s, \chi_e}$ is defined by $a \leftarrow \mathcal{U}(R_q)$, $s \leftarrow \chi_s$, $e \leftarrow \chi_e$, and $b = as + e$. The output is (a, b) . Computational problems on RLWE are defined as follows.*

1. **Search-RLWE.** *Given samples from $\text{RLWE}_{q, \chi_s, \chi_e}$, recover s .*
2. **Decision-RLWE.** *Distinguish $\text{RLWE}_{q, \chi_s, \chi_e}$ from the uniform distribution over $R_q \times R_q$.*

Definition A.2 (RLWQ). *Let $q \in \mathbb{N}$, let χ_s be a distribution over R_q , let $\Lambda \subset \mathbb{R}^n$ be a lattice, and let $\mathcal{S} \subset R$ denote the polynomial dither domain. The RLWQ distribution $\text{RLWQ}_{q, \chi_s, \Lambda}$ is defined by $a \leftarrow \mathcal{U}(R_q)$, $s \leftarrow \chi_s$, $d \leftarrow \mathcal{U}(\mathcal{S})$, and $b = \mathbf{Q}_{\Lambda, d}^\circ(as) - d$. The output is (a, b) . Computational problems on RLWQ are defined as follows.*

1. **Search-RLWQ.** *Given samples from $\text{RLWQ}_{q, \chi_s, \Lambda}$, recover s .*
2. **Decision-RLWQ.** *Distinguish $\text{RLWQ}_{q, \chi_s, \Lambda}$ from the uniform distribution over $R_q \times R_q$.*

Definition A.3 (Split-RLWQ). *Let $q \in \mathbb{N}$, let χ_s be a distribution over R_q , let $\Lambda \subset \mathbb{R}^n$ be a lattice, and let $\mathcal{S} \subset R$ denote the polynomial dither domain. The Split-RLWQ distribution $\text{SRLWQ}_{q, \chi_s, \Lambda}$ is defined by $a \leftarrow \mathcal{U}(R_q)$, $s \leftarrow \chi_s$, $d \leftarrow \mathcal{U}(\mathcal{S})$, and $c = \mathbf{Q}_{\Lambda, d}^\circ(as)$. The output is (a, c, d) . Computational problems on Split-RLWQ are defined as follows.*

1. **Search Split-RLWQ.** *Given samples from $\text{SRLWQ}_{q, \chi_s, \Lambda}$, recover s .*
2. **Decision Split-RLWQ.** *Distinguish $\text{SRLWQ}_{q, \chi_s, \Lambda}$ from the uniform distribution over $R_q \times \tau^{-1}(\Lambda \cap \mathbb{Z}_q^n) \times \mathcal{S}$.*

Theorem A.1 (Input independence of LWQ- $\mathbb{Z}$ quantization). *Let $\Delta = q/p$, let $\Lambda = \Delta\mathbb{Z}^n$, and let $K = \mathbb{Z}^n \cap \Lambda$. Let $H = \mathbb{Z}^n/K$, and let $\pi : \mathbb{Z}^n \rightarrow H$ denote the natural projection. For $\mathbf{y}, \mathbf{d} \in \mathbb{Z}^n$, define*

$$\mathbf{e}(\mathbf{y}, \mathbf{d}) \triangleq \Delta \mathbf{Q}_{\Lambda, \mathbf{d}}(\mathbf{y}) - \mathbf{d} - \mathbf{y}.$$

If $\pi(\mathbf{d})$ is uniform over H and independent of $\mathbf{y}$, then the distribution of $\mathbf{e}(\mathbf{y}, \mathbf{d})$ is independent of $\mathbf{y}$.

Proposition A.1. *Let $q = 2^\ell$, $p = 2^t$, and $\Delta = q/p$. Let $\Lambda_1 = \Delta\mathbb{Z}$ and let the scalar dither be uniform modulo Δ . Under the dither condition of Theorem A.1, the scalar quantization error distribution $\chi_{q, p}$ is the distribution of*

$$e = \Delta \left\lfloor \frac{u}{\Delta} \right\rfloor - u, \quad u \leftarrow \mathcal{U}(\{0, 1, \dots, \Delta - 1\}),$$

with the fixed nearest-rounding tie rule used by the implementation. If $\Delta = 2h$ is even, then the support is contained in $\{-h, \dots, h\}$. If $\Delta = 2h + 1$ is odd, then the support is contained in $\{-h, \dots, h\}$. In all cases $\chi_{q, p}$ is input-independent, bounded, and completely determined by Δ and the tie-breaking rule.

A.2 Centered Binomial Distributions

For a positive integer η , the centered binomial distribution $\mathcal{B}_\eta$ is defined by sampling $a_1, \dots, a_\eta, b_1, \dots, b_\eta \leftarrow \{0, 1\}$ and outputting

$$\sum_{i=1}^{\eta} (a_i - b_i).$$

When a polynomial is sampled from $\mathcal{B}_\eta$, each coefficient is sampled independently from this distribution. When a vector in R_q^k is sampled from $\mathcal{B}_\eta^k$, each coefficient of each component is sampled independently.

The concrete parameter sets use $(\eta_s, \eta_r) = (2, 2), (3, 3), (7, 7), (2, 2), (5, 5)$ for the 128-, 192-, 256-, 384-, and 512-bit profiles, respectively. The ring dimension is $n = 1024$ for MAMBA-NIKE-128, MAMBA-NIKE-192, and MAMBA-NIKE-256, and $n = 2048$ for MAMBA-NIKE-384 and MAMBA-NIKE-512, as specified in Table 3.

The deterministic implementation of `GenSecret $\eta$` consumes $2\eta \cdot n$ bits for one polynomial of degree $n - 1$. It converts the byte string to bits and forms each coefficient as the difference of two Hamming weights of length η . This is the same centered-binomial sampling pattern used in several module-lattice schemes.

A.3 Arithmetic Layer

A.3.1 Integer-Lattice Quantization

We briefly recall the integer-lattice quantizer used throughout this work. Let $q = 2^\ell$, $p = 2^t$, $0 < t < \ell$, and $\Delta = q/p = 2^{\ell-t}$. The associated integer lattice is $\Lambda = \Delta\mathbb{Z}^n$. For $\mathbf{y}, \mathbf{d} \in \mathbb{Z}^n$, the split label is computed coordinate-wise as

$$\mathbf{Q}_{\Lambda, \mathbf{d}}(\mathbf{y}) = \left\lfloor \frac{\mathbf{y} + \mathbf{d}}{\Delta} \right\rfloor \bmod p,$$

where ties are resolved by a fixed deterministic rule. If $\mathbf{c} = \mathbf{Q}_{\Lambda, \mathbf{d}}(\mathbf{y})$, its normal-form expansion is

$$\widehat{\mathbf{c}} = \Delta \mathbf{c} - \mathbf{d} \pmod{q},$$

and the corresponding quantization error is

$$\mathbf{e} = \widehat{\mathbf{c}} - \mathbf{y}.$$

When $\mathbf{d} = \mathbf{0}$, this reduces to the coefficient-wise rounding mechanism of LWR.

We transfer this quantizer to the polynomial ring through the coefficient embedding τ . For $a \in R_q$ and $d \in R$, define

$$\mathbf{Q}_{\Lambda, d}^\diamond(a) \triangleq \tau^{-1}(\mathbf{Q}_{\Lambda, \tau(d)}(\tau(a))).$$

If $c = \mathbf{Q}_{\Lambda, d}^\diamond(a)$, then the normal-form expansion is

$$\widehat{c} = \Delta c - d \pmod{q},$$

and the corresponding polynomial quantization error is

$$e = \widehat{c} - a.$$

For polynomial vectors, these operators extend component-wise.

For convenience, we associate with this quantizer a scalar reference distribution that depends explicitly on (q, p) and the fixed nearest-rounding rule. This reference distribution captures the coefficient-level shape of the quantization error. We denote it by $\chi_{q,p}$.

A.3.2 D_4 Reconciliation

We retain the NewHope-style four-dimensional reconciliation mechanism, following the reconciliation line used in Peikert’s lattice key-exchange framework and NewHope [9, 10]. Let

$$\tilde{D}_4 = \mathbb{Z}^4 \cup (\mathbf{g} + \mathbb{Z}^4), \quad \mathbf{g} = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2} \right).$$

The parameter $r_D = 2$ is used throughout the candidate profiles. For each raw key bit, the protocol groups four coefficients and sends four two-bit helper values. Thus the helper vector contributes one byte per recovered raw key bit. The submitted KEX API uses $\kappa = n/4$ consumed blocks, producing a κ -bit raw reconciliation string and an $n/4$ -byte helper vector. Accordingly, the helper is 256 bytes for the $n = 1024$ profiles and 512 bytes for the $n = 2048$ profiles.

For a polynomial $x \in R_q$, define the j -th consumed block by

$$\text{blk}_j(x) \triangleq (x_j, x_{j+\kappa}, x_{j+2\kappa}, x_{j+3\kappa}) \in \mathbb{Z}_q^4$$

for $0 \leq j < \kappa$, where $\kappa = n/4$ in the submitted implementation. All coefficients are consumed by the D_4 block partition. The initiator computes the helper h_j from $\text{blk}_j(\hat{v})$. The responder applies the same helper to $\text{blk}_j(w)$, where $w = \hat{u}s$.

The correctness condition for one block is

$$\|\text{blk}_j(\hat{v} - w)\|_1 < (1 - 2^{-r_D})q - 2.$$

For $r_D = 2$, the threshold used in the correctness estimates is

$$T_{D_4} = \left\lfloor \frac{3q}{4} \right\rfloor - 2.$$

B Security Proof Details

This subsection contains the formal passive-security statement. The main text reports the estimator outputs and attack discussion in Section 6; the notation used here follows Section 2.1.

B.1 Split and Normal RLWQ- $\mathbb{Z}$ Samples

Let $q > p \geq 1$, let $\Delta = q/p$, let $\Lambda = \Delta\mathbb{Z}^n$, and let d be a public dither satisfying the input-independence condition. For $a \leftarrow \mathcal{U}(R_q)$ and $s \leftarrow \mathcal{B}_\eta^n$, define

$$b \triangleq \mathcal{Q}_{\Lambda, d}^\diamond(as), \quad \hat{b} \triangleq \Delta b - d \pmod{q}.$$

We call (a, d, b) the split form. We call $(a, \hat{b})$ the normal form. By Theorem A.1, the normal form satisfies

$$\hat{b} = as + e,$$

where e has coefficient distribution $\chi_{q,p}$ and is independent of as in the coefficient-level model.

The public transcript of MAMBA-NIKE exposes split samples. The concrete security records in Section 6.3 use the reconstructed normal form as an RLWE proxy. This separation is important because the split transcript is the actual protocol object, while the normal projection is the object used for estimator compatibility.

B.2 Transport Samples in the KE Transcript

The responder public key contributes the split sample

$$(a, d_{\text{pk}}, b), \quad b = Q_{\Lambda_{\text{pk}}, d_{\text{pk}}}^\diamond(as_B),$$

and the one-pass message contributes the split response sample

$$(a, d_u, u), \quad u = Q_{\Lambda_u, d_u}^\diamond(ar).$$

Their normal forms are

$$\hat{b} = as_B + e_{\text{pk}}, \quad \hat{u} = ar + e_u.$$

The initiator also computes a local quantized reconciliation value whose normal form can be written as

$$\hat{v} = \hat{b}r + e_v.$$

The responder computes

$$w = \hat{u}s_B.$$

The resulting agreement error is

$$\hat{v} - w = e_{\text{pk}}r - e_us_B + e_v,$$

which is the quantity controlled by the D_4 reconciliation and key-acceptance analysis.

B.3 Passive Real-or-Random Security of MAMBA-NIKE

Theorem B.1 (Direct passive real-or-random security). *Assume that:*

1. *the public-key split sample is decisional split-RLWQ pseudorandom;*
2. *the one-pass response split sample is decisional split-RLWQ pseudorandom;*
3. *the reconciliation leakage is bounded by ε_{rec} ; and*
4. *$\text{KDF}(\nu, \text{pk}_B, M_1)$ is modeled as a transcript-bound random oracle output, instantiated by SHAKE256 in the implementation.*

Then for every probabilistic polynomial-time passive adversary $\mathcal{A}$ against the one-pass KE experiment, there exist probabilistic polynomial-time distinguishers $\mathcal{B}_{\text{pk}}$ and $\mathcal{B}_u$ such that

$$\text{Adv}_{\text{MAMBA-NIKE}}^{\text{ror}}(\mathcal{A}) \leq \text{Adv}_{\text{sRLWQ}}^{\text{dec}}(\mathcal{B}_{\text{pk}}; q, p_{\text{pk}}, \mathcal{B}_{\eta_s}) + \text{Adv}_{\text{sRLWQ}}^{\text{dec}}(\mathcal{B}_u; q, p_u, \mathcal{B}_{\eta_r}) + \varepsilon_{\text{rec}} + \rho_{\text{key}}.$$

The term ρ_{key} accounts for executions in which the responder does not recover the initiator's raw key.

Proof. We use a sequence of games over the public transcript

$$T = (\text{pk}_B, M_1) = ((\rho, b), (\mu, u, h)).$$

Let ℓ_{ss} denote the declared shared-secret length in bits for the selected profile.

Game G_0 . This is the real passive experiment. The challenger generates

$$\text{pk}_B = (\rho, b), \quad M_1 = (\mu, u, h),$$

computes the initiator raw reconciliation string ν , and returns

$$K = \text{KDF}(\nu, \text{pk}_B, M_1)$$

or a uniform ℓ_{ss} -bit string according to the hidden challenge bit.

Game G_1 . Replace the responder public-key split sample (a, d_{pk}, b) by a uniformly random split sample with the same public seed and label space. Any adversary distinguishing G_0 from G_1 gives a distinguisher $\mathcal{B}_{\text{pk}}$ against the decisional split-RLWQ assumption for the public-key layer. Thus

$$|\Pr[G_0 \Rightarrow 1] - \Pr[G_1 \Rightarrow 1]| \leq \text{Adv}_{\text{sRLWQ}}^{\text{dec}}(\mathcal{B}_{\text{pk}}; q, p_{\text{pk}}, \mathcal{B}_{\eta_s}).$$

Game G_2 . Replace the one-pass response split sample (a, d_u, u) by a uniformly random response split sample with the same public dither seed μ and label space. Any adversary distinguishing G_1 from G_2 gives a distinguisher $\mathcal{B}_u$ against the decisional split-RLWQ assumption for the response layer. Thus

$$|\Pr[G_1 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq \text{Adv}_{\text{sRLWQ}}^{\text{dec}}(\mathcal{B}_u; q, p_u, \mathcal{B}_{\eta_r}).$$

Game G_3 . In G_2 , the remaining public information correlated with the raw reconciliation string is the helper h and the fixed transcript formatting. By Definition 6.2, conditioned on the transcript, the raw reconciliation string differs from uniform by at most ε_{rec} in statistical distance. Replacing ν by a uniform κ -bit string therefore changes the adversary's success probability by at most ε_{rec} .

Game G_4 . With uniform ν , the transcript-bound random-oracle value

$$\text{KDF}(\nu, \text{pk}_B, M_1)$$

is distributed as a uniform ℓ_{ss} -bit session key from the adversary's passive view. Hence G_4 is exactly the random-key experiment.

Combining the three game hops by the triangle inequality gives the indistinguishability part of the stated bound. If the experiment additionally requires the responder-side key K_B to match the initiator key K_A , then executions in which reconciliation rejects are charged separately, contributing the additive ρ_{key} term. $\square$

B.4 Scope of Active and CCA-Style Claims

The theorem above is intentionally a direct passive KE proof. It does not proceed by proving a PKE or KEM IND-CPA theorem and then deriving KE security. That modular PKE/KEM route is appropriate when the online message is naturally a ciphertext or encapsulation and the receiver performs decryption or decapsulation. Here the online message is a reconciliation transcript, and the responder derives the key by reconstructing a close algebraic value, not by decrypting a ciphertext.

The standalone MAMBA-NIKE interface does not expose a decryption or decapsulation oracle. Accordingly, this document does not claim IND-CCA security, authenticated key exchange security, active-attack security, identity binding, replay protection, UKS/KCI resistance, or channel binding. Those properties must be supplied by a higher-level protocol if required. The distinction between this passive, unauthenticated experiment and active authenticated key-exchange models is consistent with the standard separation of adversarial capabilities in key-exchange security definitions [14, 15].

C Key-Acceptance Error Analysis

C.1 Correctness and Key-Acceptance Analysis

The reconstructed polynomials satisfy

$$\hat{b} = as + e_{\text{pk}}, \quad \hat{u} = ar + e_u, \quad \hat{v} = \hat{b}r + e_v.$$

The initiator computes $\hat{v}$. The responder computes $w = \hat{u}s$. Their difference is

$$\Delta \triangleq \hat{v} - w = (as + e_{\text{pk}})r + e_v - (ar + e_u)s = e_{\text{pk}}r - e_us + e_v.$$

This is the only error expression needed for reliability analysis. The terms e_{pk} , e_u , and e_v follow the scalar reference distributions χ_{pk} , χ_u , and χ_v coefficient-wise under the independent-coefficient model.

For each consumed D_4 block, define the key-acceptance event

$$R_j = \{\|\text{blk}_j(\Delta)\|_1 < T_{D_4}\}, \quad T_{D_4} = \lfloor 3q/4 \rfloor - 2.$$

The whole-key acceptance probability and key-rejection probability are

$$P_{\text{acc}} = \Pr \left[\bigcap_{j=0}^{\kappa-1} R_j \right], \quad \rho_{\text{key}} = 1 - P_{\text{acc}},$$

where $\kappa = n/4$ is the number of consumed D_4 blocks in the submitted API.

Table 11: Correctness verification records for the submitted MAMBA-NIKE profiles. The observed key-rejection rate is computed from the repository benchmark records; all five profiles completed 5,000 agreements with zero observed failures. The final column reproduces the current rejection status.

Instance	q	t_{pk}	t_u	T_{D_4}	Trials	$\hat{P}_{\text{acc}}$	Numerical status
MAMBA-NIKE-128	2^{13}	9	10	6142	5,000	1.0000	$2^{-42.44}$
MAMBA-NIKE-192	2^{13}	10	10	6142	5,000	1.0000	$2^{-40.65}$
MAMBA-NIKE-256	2^{13}	10	10	6142	5,000	1.0000	$2^{-39.97}$
MAMBA-NIKE-384	2^{13}	11	11	6142	5,000	1.0000	$2^{-39.69}$
MAMBA-NIKE-512	2^{13}	11	11	6142	5,000	1.0000	$2^{-37.81}$

Under the current p-packed byte accounting, MAMBA-NIKE-128 has `Pass_Num` = 1, `M1_Len` = 1568, and `SS_Len` = 32. The empirical records in Table 11 document current conformance evidence only. A certified cryptographic upper bound requires a numerically verified method with an error budget below the relevant tail probabilities.

C.1.1 Exact Decoding Radius

The recovery map operates over the D_4 lattice and is analyzed block-wise. For $r_D = 2$, define $T_{D_4} = \lfloor 3q/4 \rfloor - 2$.

Lemma C.1. *Let $w_{\text{help}}, w_{\text{rec}} \in R_q$, let $h \leftarrow \text{HelpRec}(w_{\text{help}})$, and let $\nu \leftarrow \text{Rec}(w_{\text{help}}, h)$. If for every consumed block j ,*

$$\|\text{blk}_j(w_{\text{help}} - w_{\text{rec}})\|_1 < T_{D_4},$$

then

$$\text{Rec}(w_{\text{rec}}, h) = \nu.$$

Proof. This is the standard NewHope/Peikert D_4 reconciliation correctness condition. The helper determines a block decision region; an ℓ_1 perturbation smaller than T_{D_4} cannot cross the opposite decision boundary. Applying this argument to each consumed block yields the claim. $\square$

C.1.2 A Zero-Failure Sufficient Condition

Let

$$B_s \triangleq \max\{|t| : \chi_s(t) > 0\}, \quad B_r \triangleq \max\{|t| : \chi_r(t) > 0\}, \\ B_{\text{pk}} \triangleq \max\{|t| : \chi_{\text{pk}}(t) > 0\}, \quad B_u \triangleq \max\{|t| : \chi_u(t) > 0\}, B_v \triangleq \max\{|t| : \chi_v(t) > 0\},$$

where χ_{pk} and χ_u denote the coefficient distributions of the two transport quantization errors. For every coefficient index i , the coefficient $\tau(\Delta)_i$ is a signed sum of n products of the form $\tau(e_{\text{pk}})_j \tau(r)_{i-j}$ and $\tau(s)_j \tau(e_u)_{i-j}$. Hence one always has the deterministic support bound

$$|\tau(\Delta)_i| \leq n(B_{\text{pk}}B_r + B_sB_u) + B_v.$$

Proposition C.1. *If $4(n(B_{\text{pk}}B_r + B_sB_u) + B_v) < T_{D_4}$ then the protocol has unit key-acceptance probability.*

Proof. Under the stated inequality, the ℓ_1 norm of every four-coefficient agreement-error block is strictly smaller than T_{D_4} . Lemma C.1.1 therefore applies to all consumed blocks. Hence the two parties recover the same raw key. $\square$

Proposition C.1 is conservative, but it isolates a useful design principle. The reconciliation radius must dominate the worst-case amplification of the transport errors by the secret distributions. When this is not possible under the desired bandwidth, one must analyze the exact tail of the agreement error instead of relying on the support bound.

C.1.3 Computable Key-Rejection Upper Bound

Define the scalar product distributions

$$\psi_{\text{pk},r}(t) \triangleq \Pr[E_{\text{pk}}R = t], \quad \psi_{s,u}(t) \triangleq \Pr[SE_u = t]$$

where $E_{\text{pk}} \leftarrow \chi_{\text{pk}}$, $E_u \leftarrow \chi_u$, $S \leftarrow \chi_s$, and $R \leftarrow \chi_r$ are independent scalar variables.

If the scalar laws are symmetric, then the sign pattern induced by negacyclic multiplication does not change these distributions, and one obtains

$$\text{Law}(\tau(\Delta)_i) = \psi_{\text{pk},r}^{(*n)} * \tilde{\psi}_{s,u}^{(*n)} * \chi_v$$

where $\tilde{\psi}_{s,u}(t) \triangleq \psi_{s,u}(-t)$.

If the scalar laws are not symmetric, the exact coefficient law is obtained by incorporating the negacyclic signs directly into the single-term distributions before taking the n -fold convolution. In either case, the exact coefficient distribution is computable by dynamic programming or FFT-based convolution.

Accordingly, the key-rejection probability satisfies the union bound

$$\rho_{\text{key}} \leq \sum_{j=0}^{\kappa-1} \Pr[\|\text{blk}_j(\Delta)\|_1 \geq T_{D_4}].$$

If consumed blocks are bounded with a common distribution, this simplifies to

$$\rho_{\text{key}} \leq \kappa \cdot \Pr\left[\sum_{\ell=0}^3 |\Delta_\ell| \geq T_{D_4}\right],$$

where $\kappa = n/4$ is the number of consumed D_4 blocks. Thus a computable analytic lower bound on key acceptance is

$$P_{\text{acc}} \geq 1 - \rho_{\text{key}}^{\text{ub}}, \quad \rho_{\text{key}}^{\text{ub}} = \kappa \cdot \Pr \left[\sum_{\ell=0}^3 |\Delta_\ell| \geq T_{D_4} \right],$$

with the probability evaluated from the exact induced coefficient distribution or from the exact joint block distribution when block correlations are retained.

This is the preferred form for concrete key-acceptance estimation. It uses the exact induced quantization error distributions rather than a surrogate Gaussian approximation. It also makes the contribution of the design levers explicit: Λ_{pk} and Λ_u determine χ_{pk} and χ_u , Λ_v determines χ_v , and the secret distributions χ_s and χ_r determine the amplification mechanism.

C.1.4 Wrap-Around Separation

The exact formulation above is the right one for the integer coefficient domain, but one further point is kept separate in the full proof. The identity $\Delta = e_{\text{pk}}r - e_us + e_v$ is interpreted coefficient-wise through centered representatives. Accordingly, the analysis separates the event that a coefficient of the centered representative wraps around modulo q from the event that a D_4 block crosses its reconciliation boundary.

Let

$$\rho_{\text{wrap}} \triangleq \Pr [\exists i \in \{0, \dots, n-1\} : |\tau(\Delta)_i| \geq q/2]$$

and let

$$\rho_{\text{rec}} \triangleq \Pr [\exists j : \|\text{blk}_j(\Delta)\|_1 \geq T_{D_4}].$$

Then the total key-rejection probability satisfies

$$\rho_{\text{key}} \leq \rho_{\text{wrap}} + \rho_{\text{rec}}.$$

In well-chosen parameter sets, the reconciliation-error tail lies far below $q/2$ and ρ_{wrap} is typically negligible compared with the reconciliation-boundary term. Nevertheless, keeping the two events separate is important for a fully precise argument in the integer coefficient model.

References

- [1] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In *Advances in Cryptology – EUROCRYPT 2012*, pages 719–737. Springer, 2012.
- [2] Shanxiang Lyu, Ling Liu, and Cong Ling. Learning With Quantization: A Ciphertext Efficient Lattice Problem with Tight Security Reduction from LWE. *Cryptology ePrint Archive*, Paper 2024/714, 2024. <https://eprint.iacr.org/2024/714>.
- [3] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In *Progress in Cryptology – AFRICACRYPT 2018*, pages 282–305. Springer, 2018.
- [4] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

- [5] Nicolas Gama and Phong Q. Nguyen. Predicting lattice reduction. In *Advances in Cryptology – EUROCRYPT 2008*, pages 31–51. Springer, 2008.
- [6] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. In *Advances in Cryptology – ASIACRYPT 2011*, pages 1–20. Springer, 2011.
- [7] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6), Article 34, 2009.
- [8] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology – EUROCRYPT 2010*, pages 1–23. Springer, 2010.
- [9] Chris Peikert. Lattice cryptography for the Internet. In *Post-Quantum Cryptography – PQCrypto 2014*, pages 197–219. Springer, 2014.
- [10] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange – a new hope. In *25th USENIX Security Symposium*, pages 327–343, 2016.
- [11] National Institute of Standards and Technology. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Federal Information Processing Standards Publication 202, 2015.
- [12] Andrei L. Toom. The complexity of a scheme of functional elements realizing the multiplication of integers. *Soviet Mathematics Doklady*, 3:714–716, 1963.
- [13] Stephen A. Cook. On the minimum computation time of functions. PhD thesis, Harvard University, 1966.
- [14] Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In *Advances in Cryptology – CRYPTO ’93*, volume 773 of *Lecture Notes in Computer Science*, pages 232–249. Springer, 1994.
- [15] Ran Canetti and Hugo Krawczyk. Analysis of key-exchange protocols and their use for building secure channels. In *Advances in Cryptology – EUROCRYPT 2001*, volume 2045 of *Lecture Notes in Computer Science*, pages 453–474. Springer, 2001.
- [16] Richard Lindner and Chris Peikert. Better key sizes (and attacks) for LWE-based encryption. In *Topics in Cryptology – CT-RSA 2011*, volume 6558 of *Lecture Notes in Computer Science*, pages 319–339. Springer, 2011.
- [17] Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the LWE, NTRU schemes! In *Security and Cryptography for Networks – SCN 2018*, volume 11035 of *Lecture Notes in Computer Science*, pages 351–367. Springer, 2018.
- [18] Martin R. Albrecht, Léo Ducas, Gottfried Herold, Elena Kirshanova, Eamonn W. Postlethwaite, and Marc Stevens. The general sieve kernel and new records in lattice reduction. In *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 717–746. Springer, 2019.
- [19] Martin R. Albrecht, Vlad Gheorghiu, Eamonn W. Postlethwaite, and John M. Schanck. Estimating quantum speedups for lattice sieves. In *Advances in Cryptology – ASIACRYPT 2020, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 583–613. Springer, 2020.

- [20] Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. On the impact of decryption failures on the security of LWE/LWR-based schemes. *Cryptology ePrint Archive*, Paper 2018/1089, 2018. <https://eprint.iacr.org/2018/1089>.
- [21] Jan-Pieter D’Anvers, Mélissa Rossi, and Fernando Virdia. (One) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 3–33. Springer, 2020.
- [22] Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. LWE with side information: Attacks and concrete security estimation. In *Advances in Cryptology – CRYPTO 2020, Part II*, volume 12171 of *Lecture Notes in Computer Science*, pages 329–358. Springer, 2020.
- [23] Paul C. Kocher. Timing attacks on implementations of Diffie–Hellman, RSA, DSS, and other systems. In *Advances in Cryptology – CRYPTO ’96*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.
- [24] Dag Arne Osvik, Adi Shamir, and Eran Tromer. Cache attacks and countermeasures: The case of AES. In *Topics in Cryptology – CT-RSA 2006*, volume 3860 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2006.
- [25] Yuval Yarom and Katrina Falkner. FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In *23rd USENIX Security Symposium*, pages 719–732. USENIX Association, 2014.
- [26] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(3):307–335, 2020.
- [27] Mihir Bellare, Hannah Davis, and Felix Günther. Separate your domains: NIST PQC KEMs, oracle cloning, and read-only indistinguishability. In *Advances in Cryptology – EUROCRYPT 2020, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 3–32. Springer, 2020.
- [28] Jeff Hoffstein, Jill Pipher, John M. Schanck, Joseph H. Silverman, William Whyte, and Zhenfei Zhang. Choosing parameters for NTRUEncrypt. In *Topics in Cryptology – CT-RSA 2017*, volume 10159 of *Lecture Notes in Computer Science*, pages 3–18. Springer, 2017.
- [29] Ram Zamir. *Lattice Coding for Signals and Networks*. Cambridge University Press, 2014.